

```

"""
unit_leaddata.py – Dynamic Multi-Source Router (Registry Architecture)
"""
import logging
import importlib
from pathlib import Path
from typing import Dict, Any
from cf2.meta import mark_subtask
from cf2.core.paths import RUNTIME_PATHS
from cf2.utils.leaddata_geo_resolver import resolve_global_context

# Standard Pipeline Tools (Always run)
from cf2.tools.leaddata_normalize import LeadDataNormalizeTool
from cf2.tools.leaddata_score import LeadDataScoreTool
from cf2.tools.leaddata_enrich_osint import OSINTEnrichTool
from cf2.tools.leaddata_export import LeadDataExportTool

logger = logging.getLogger(__name__)

# =====
# 🗺️ THE PLUGIN REGISTRY
# Maps YAML strings to Python Module Paths.
# =====

TOOL_REGISTRY = {
    # --- Location Sources ---
    "google_maps": {"module": "cf2.tools.leaddata_collect", "class":
"LeadDataCollectTool", "args": {"sources": ["maps"]}},
    "maps_reviewers": {"module": "cf2.tools.leaddata_collect", "class":
"LeadDataCollectTool", "args": {"sources": ["maps_reviewers"]}},
    "yelp": {"module": "cf2.tools.leaddata_yelp", "class":
"YelpScraperTool", "args": {}}, # Coming soon
    "bing_places": {"module": "cf2.tools.leaddata_bing", "class":
"BingPlacesTool", "args": {}}, # Coming soon

    # --- Directory Sources (B2B Gold) ---
    "linkedin_company": {"module": "cf2.tools.leaddata_linkedin", "class":
"LinkedInScraperTool", "args": {}}, # ✅ NEW
    "clutch": {"module": "cf2.tools.leaddata_clutch", "class":
"ClutchScraperTool", "args": {}}, # Coming soon
    "crunchbase": {"module": "cf2.tools.leaddata_crunchbase", "class":
"CrunchbaseTool", "args": {}}, # Coming soon

    # --- Community / Intent Sources ---
    "reddit_travel": {"module": "cf2.tools.leaddata_reddit", "class":
"RedditTravelScraperTool", "args": {}},
    "reddit_business": {"module": "cf2.tools.leaddata_intent", "class":
"LeadDataIntentTool", "args": {"force_reddit": True}},
    "quora": {"module": "cf2.tools.leaddata_intent", "class":
"LeadDataIntentTool", "args": {"force_quora": True}},

    # --- Trend Sources ---
    "youtube_comments": {"module": "cf2.tools.leaddata_youtube", "class":
"YouTubeTrendTool", "args": {}}, # Coming soon
    "google_trends": {"module": "cf2.tools.leaddata_news", "class":
"GoogleTrendsTool", "args": {}}, # ✅ NEW
}

def _parse_keywords(topic: str) -> list:
    return [k.strip() for k in topic.split(",") if k.strip()]

def _run_step(workspace: Path, unit_name: str, step_name: str, tool_class: type,

```

```

kwargs: dict) -> str:
    """Generic Tool Executor."""
    try:
        logger.info(f"🌀 [{step_name}] Starting...")
        result = tool_class()._run(**kwargs)
        logger.info(f"✅ [{step_name}] {result}")
        mark_subtask(workspace, unit_name, step_name, "done")
        return "done"
    except Exception as e:
        logger.warning(f"⚠️ [{step_name}] Skipped: {e}")
        return "failed"

def run(topic: str, workspace: Path, inputs: dict, force: bool = False) -> str:
    try:
        workspace = Path(workspace)
        leaddata_dir = workspace / "leaddata"
        leaddata_dir.mkdir(parents=True, exist_ok=True)
        unit_name = "Unit-LeadData"

        cfg = inputs.get("leaddata_config", {})
        if not cfg.get("enabled", True): return "disabled"

        # — Setup Context —————
        keywords = _parse_keywords(topic)
        geo = resolve_global_context(topic, cfg)
        out_dir = str(leaddata_dir)

        # Resolve API Keys
        creds_file = cfg.get("credentials_file", "")
        if creds_file and not Path(creds_file).is_absolute():
            creds_file = str(RUNTIME_PATHS["secrets"] / Path(creds_file).name)

        # Base kwargs passed to ALL registry tools
        base_kwargs = {
            "topic": topic,
            "keywords": keywords,
            "output_dir": out_dir,
            "credentials_file": creds_file,
            "skip_if_cached": cfg.get("skip_if_cached", True)
        }

        # =====
        # PHASE 1: DYNAMIC SOURCE INGESTION (The Registry Loop)
        # =====
        logger.info(f"🚀 Phase 1: Source Ingestion")

        # Merge the 4 categorized arrays from YAML
        requested_sources = (
            cfg.get("location_sources", []) +
            cfg.get("directory_sources", []) +
            cfg.get("community_sources", []) +
            cfg.get("trend_sources", [])
        )

        tool_overrides = cfg.get("tool_overrides", {})

        for source_name in requested_sources:
            reg = TOOL_REGISTRY.get(source_name)

            if not reg:
                logger.warning(f"❓ Source '{source_name}' not in TOOL_REGISTRY.
Skipping.")
                continue

```

```

try:
    # Dynamically import the tool class
    module = importlib.import_module(reg["module"])
    tool_class = getattr(module, reg["class"])

    # Merge args: Base kwargs + Registry defaults + YAML overrides
    final_kwargs = {**base_kwargs, **reg.get("args", {})},
**tool_overrides.get(source_name, {}))

    # Inject dynamic geo rules if the tool accepts them
    varnames = tool_class._run.__code__.co_varnames
    if "phone_country_prefix" in varnames:
        final_kwargs["phone_country_prefix"] =
tool_overrides.get(source_name, {}).get("phone_country_prefix") or geo["prefix"]
    if "phone_country_default" in varnames:
        final_kwargs["phone_country_default"] =
tool_overrides.get(source_name, {}).get("phone_country_default") or geo["code"]
    if "hl" in varnames:
        final_kwargs["hl"] = tool_overrides.get(source_name,
{}).get("hl") or geo["hl"]

    _run_step(workspace, unit_name, source_name.upper(), tool_class,
final_kwargs)

except ImportError:
    logger.info(f"🔧 [{source_name.upper()}] Tool file not created
yet. Coming soon.")
except Exception as e:
    logger.error(f"❌ [{source_name.upper()}] Unexpected error:
{e}")

# =====
# PHASE 2: STANDARD PROCESSING PIPELINE
# =====
logger.info(f"🧠 Phase 2: Standard Processing")

# 1. Normalize
norm_cfg = cfg.get("normalize_config", {})
_run_step(workspace, unit_name, "Normalize", LeadDataNormalizeTool, {
    "output_dir": out_dir,
    "deduplicate_on": norm_cfg.get("deduplicate_on", ["name"]),
    "phone_country_default": norm_cfg.get("phone_country_default") or
geo["code"],
    "lowercase_email": norm_cfg.get("lowercase_email", True),
    "force_https": norm_cfg.get("force_https", True),
    "strip_unicode": norm_cfg.get("strip_unicode", True),
    "min_name_length": norm_cfg.get("min_name_length", 2),
})

# 2. Score
score_cfg = cfg.get("score_config", {})
_run_step(workspace, unit_name, "Score", LeadDataScoreTool, {
    "output_dir": out_dir,
    "score_enabled": score_cfg.get("score_enabled", True),
    "scoring_rubric": score_cfg.get("scoring_rubric", {"source": 50,
"intent_score": 30, "review_count": 15, "review_date": 10}),
    "thresholds": score_cfg.get("segment_thresholds", {"hot": 60,
"warm": 35, "cold": 0}),
    "sort_by_score_desc": score_cfg.get("sort_by_score_desc", True),
})

# 3. Enrich (Optional OSINT)
if cfg.get("enrich_enabled", False):

```

```

        enrich_cfg = cfg.get("enrich_config", {})
        _run_step(workspace, unit_name, "Enrich", OSINTEnrichTool, {
            "input_file": str(leaddata_dir / "scored" / "leads_scored.csv"),
            "output_dir": out_dir,
            "credentials_file": creds_file,
            "min_confidence": enrich_cfg.get("min_confidence", 0.30),
            "allow_guessing": enrich_cfg.get("allow_guessing", False),
            "max_osint_queries": enrich_cfg.get("max_osint_queries", 15),
        })

    # 4. Export
    export_cfg = cfg.get("export_config", {})
    _run_step(workspace, unit_name, "Export", LeadDataExportTool, {
        "output_dir": out_dir,
        "formats": export_cfg.get("formats", ["csv", "json"]),
        "generate_stats": export_cfg.get("generate_stats", True),
        "stats_file": export_cfg.get("stats_file", "lead_stats.json"),
        "include_segments_breakdown":
export_cfg.get("include_segments_breakdown", True),
    })

    logger.info(f"✅ Done: {leaddata_dir}")
    return "done"

except Exception as e:
    logger.error(f"❌ Unit-LeadData failed critically: {e}")
    return "failed"

```

```

=====
=
"""
leaddata_collect.py – Dynamic Global Lead Collection Tool (CF2 Compliant)
Output: {output_dir}/raw/leads_raw.csv

Sources (switchable via config):
    maps:                Fetch BUSINESS listings via SerpAPI Google Maps (Global)
                        → Fields: name, phone, phone_formatted, address, website,
location
    maps_reviewers:      Fetch PEOPLE who reviewed hotels/attractions (traveler
mining)
                        → Fields: name, location, review_date, hotel_reviewed,
review_count
    csv:                 Optional fallback for testing only

Global/Dynamic Features:
    - Accepts 'hl' parameter for localized Google Maps results (e.g., 'fr' for
France, 'de' for Germany).
    - Dynamic phone prefixing (+1, +44, +33, etc.) passed from the router.
    - Optional lightweight email scraper (scrape_emails=True).
"""

from pathlib import Path
from typing import Type, List, Dict, Any, Tuple
import csv
import json
import re
import time
import requests
from datetime import datetime
from pydantic import BaseModel, Field
from crewai.tools import BaseTool

```

```

SERPAPI_ENDPOINT = "https://serpapi.com/search.json"

# Strict universal schema expected by downstream normalize/score tools
CSV_FIELDS = [
    # Core identity
    "name", "phone", "phone_formatted", "email", "website",
    "address", "location",
    # Classification
    "category", "source", "keyword",
    # Quality signals
    "rating", "review_count", "review_snippet",
    # Traveler-specific (empty for business leads)
    "destination_visited", "review_date", "hotel_reviewed",
    # Scoring pipeline
    "intent_score", "quality_score", "segment",
    # Metadata
    "last_verified",
]

SUPPORTED_SOURCES = {"maps", "maps_reviewers", "csv"}

# -----
# Credential helpers
# -----

def _load_api_key(credentials_file: str) -> str:
    """Read api_key from JSON credentials file."""
    if not credentials_file:
        return ""
    p = Path(credentials_file).expanduser()
    if not p.exists():
        return ""
    try:
        data = json.loads(p.read_text(encoding="utf-8"))
        key = data.get("api_key")
        if key:
            return str(key).strip()
        for k in data.get("keys", []):
            if k.get("status") == "active" and k.get("api_key"):
                return str(k["api_key"]).strip()
        return ""
    except Exception:
        return ""

# -----
# Data cleaning & formatting helpers
# -----

def _today() -> str:
    return datetime.now().strftime("%Y-%m-%d")

def _clean_text(value: Any, max_len: int = None) -> str:
    if value is None:
        return ""
    if isinstance(value, dict):
        value = (
            value.get("text") or value.get("snippet")
            or value.get("description") or value.get("value") or ""
        )
    text = str(value).replace("\n", " ").replace("\r", " ").strip()
    text = re.sub(r"\s+", " ", text)

```

```

return text[:max_len] if max_len is not None else text

def _to_int(value: Any) -> int:
    if value is None: return 0
    if isinstance(value, bool): return int(value)
    if isinstance(value, (int, float)): return int(value)
    if isinstance(value, dict):
        for key in ["reviews", "review_count", "count", "total", "value",
"text"]]:
            if key in value:
                parsed = _to_int(value[key])
                if parsed: return parsed
        return 0
    s = str(value).lower().replace(",", "").strip()
    if not s: return 0
    m = re.search(r"(\d+(?:\.\d+)?)\s*([kmb])?\s*reviews?", s)
    if not m: m = re.search(r"(\d+(?:\.\d+)?)\s*([kmb])?", s)
    if not m: return 0
    n = float(m.group(1))
    suffix = (m.group(2) or "").lower()
    multipliers = {"k": 1_000, "m": 1_000_000, "b": 1_000_000_000}
    return int(n * multipliers.get(suffix, 1))

def _to_float(value: Any) -> float:
    if value is None: return 0.0
    try: return float(value)
    except (ValueError, TypeError): return 0.0

def _category_to_str(value: Any, default: str = "Business") -> str:
    if value is None: return default
    if isinstance(value, list):
        cleaned = [_clean_text(v) for v in value if _clean_text(v)]
        return ", ".join(cleaned) if cleaned else default
    text = _clean_text(value)
    return text or default

def _format_phone_e164(phone: str, country_prefix: str = "+1") -> str:
    """Format phone to E.164 standard using dynamic country prefix."""
    if not phone: return ""
    cleaned = re.sub(r"\s+", "", phone.strip())
    cleaned = re.sub(r"^\d+", "", cleaned)
    if not cleaned: return ""
    if not cleaned.startswith("+"):
        cleaned = cleaned.lstrip("0")
        cleaned = country_prefix + cleaned
    return cleaned

def _phone_is_valid(phone: str) -> bool:
    """Basic check: at least 7 digits."""
    return len(re.sub(r"\D", "", phone)) >= 7

def _scrape_email_from_website(website: str, timeout: int = 5) -> str:
    """Lightweight, fast scrape to find a contact email on a homepage."""
    if not website: return ""
    try:
        if not website.startswith("http"): website = f"https://{website}"
        r = requests.get(website, timeout=timeout, headers={"User-Agent":
"Mozilla/5.0 (Windows NT 10.0; Win64; x64)"}))
        if r.status_code == 200:
            emails = re.findall(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', r.text)
            valid = [e for e in emails if not any(x in e.lower() for x in
['wix.com', 'sentry', 'googleapis', 'wordpress.org', 'example.com'])]
            if valid: return valid[0].lower()

```

```

except Exception:
    pass
return ""

# -----
# SerpAPI response parsers
# -----

def _get_local_results(payload: Dict[str, Any]) -> List[Dict[str, Any]]:
    if not isinstance(payload, dict): return []
    for key in ["local_results", "places_results"]:
        value = payload.get(key)
        if isinstance(value, list): return value
        if isinstance(value, dict):
            for subkey in ["places", "results", "items"]:
                subvalue = value.get(subkey)
                if isinstance(subvalue, list): return subvalue
    return []

def _get_reviews_from_payload(payload: Dict[str, Any]) -> List[Dict[str, Any]]:
    if not isinstance(payload, dict): return []
    value = payload.get("reviews")
    if isinstance(value, list): return value
    for key in ["reviews_results", "user_reviews"]:
        value = payload.get(key)
        if isinstance(value, list): return value
        if isinstance(value, dict):
            for subkey in ["reviews", "results", "items"]:
                subvalue = value.get(subkey)
                if isinstance(subvalue, list): return subvalue
    return []

def _place_identifier_params(place: Dict[str, Any]) -> Tuple[Dict[str, str], str]:
    data_id = place.get("data_id")
    place_id = place.get("place_id")
    if data_id: return {"data_id": data_id}, "data_id"
    if place_id: return {"place_id": place_id}, "place_id"
    return {}, ""

# -----
# Date parsing (traveler mode)
# -----

def _parse_review_date(date_str: str, recency_days: int = 90) -> bool:
    if not date_str: return True
    s = str(date_str).lower().strip()
    s = re.sub(r"^edited\s+", "", s).strip()
    if not s: return True
    if any(token in s for token in ["just now", "today"]): return True
    if "yesterday" in s: return recency_days >= 1
    m = re.search(r"(\d+|a|an)\s*(second|minute|hour|day|week|month|year)s?\s+ago", s)
    if m:
        num_str, unit = m.groups()
        num = 1 if num_str in {"a", "an"} else int(num_str)
        days_ago = {"second": 0, "minute": 0, "hour": 0, "day": num, "week": num
* 7, "month": num * 30, "year": num * 365}.get(unit, 999999)
        return days_ago <= recency_days
    for fmt in ["%B %Y", "%b %Y", "%B %d, %Y", "%b %d, %Y", "%Y-%m-%d",
"%m/%d/%Y", "%d/%m/%Y"]:
        try:

```

```

        review_dt = datetime.strptime(s, fmt)
        return (datetime.now() - review_dt).days <= recency_days
    except ValueError: continue
    except Exception: return True
return True

def _get_user_review_count(user: Dict[str, Any]) -> int:
    if not isinstance(user, dict): return 0
    candidates = [user.get("reviews"), user.get("review_count"),
user.get("review_count_text"), user.get("contributor_reviews"),
user.get("contributions"), user.get("activity")]
    for c in candidates:
        parsed = _to_int(c)
        if parsed: return parsed
    return 0

# -----
# CSV I/O
# -----

def _csv_has_data_rows(path: Path) -> bool:
    if not path.exists() or path.stat().st_size == 0: return False
    try:
        with open(path, newline="", encoding="utf-8") as f:
            reader = csv.reader(f)
            next(reader, None)
            for row in reader:
                if any(str(cell).strip() for cell in row): return True
            return False
    except Exception: return path.stat().st_size > 0

def _write_leads_csv(path: Path, leads: List[Dict[str, Any]]) -> None:
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=CSV_FIELDS, extrasaction="ignore",
restval="")
        w.writeheader()
        w.writerows(leads)

# -----
# Pydantic schema
# -----

class LeadDataCollectInput(BaseModel):
    topic: str = Field(...)
    keywords: List[str] = Field(...)
    output_dir: str = Field(...)
    sources: List[str] = Field(default_factory=lambda: ["maps"])
    credentials_file: str = Field(default="")
    api_endpoint: str = Field(default=SERPAPI_ENDPOINT)
    engine: str = Field(default="google_maps")
    search_type: str = Field(default="search")
    hl: str = Field(default="en") # DYNAMIC: Host language for localized
results
    request_timeout: int = Field(default=30)
    max_results_per_keyword: int = Field(default=20)
    skip_if_cached: bool = Field(default=True)
    # — Business-mode params
    require_phone: bool = Field(default=True)
    require_website: bool = Field(default=False)
    min_rating: float = Field(default=0.0)
    min_reviews: int = Field(default=0)
    phone_country_prefix: str = Field(default="+1") # DYNAMIC: Passed from

```



```

router
    request_delay_seconds: float = Field(default=1.0)
    scrape_emails: bool = Field(default=False) # Optional global enrichment
    # — Traveler-mode params —————
    reviewer_recency_days: int = Field(default=90)
    reviewer_min_activity: int = Field(default=5)
    # — CSV fallback —————
    csv_file: str = Field(default="")

# —————
# Tool
# —————

class LeadDataCollectTool(BaseTool):
    name: str = "leaddata_collect"
    description: str = (
        "Dynamically collect raw global leads from SerpAPI Google Maps. "
        "source='maps' → business leads. source='maps_reviewers' → traveler
leads. "
        "Output: raw/leads_raw.csv"
    )
    args_schema: Type[BaseModel] = LeadDataCollectInput

    def _collect_maps_businesses(
        self, api_key: str, keywords: List[str], api_endpoint: str, engine: str,
        search_type: str, hl: str, request_timeout: int,
max_results_per_keyword: int,
        require_phone: bool, require_website: bool, min_rating: float,
float,
        min_reviews: int, phone_country_prefix: str, request_delay_seconds:
float,
        scrape_emails: bool
    ) -> List[Dict[str, Any]]:
        """Collect business listings globally from Google Maps."""
        all_leads: List[Dict[str, Any]] = []
        seen: set = set()

        for kw in keywords:
            try:
                print(f" 🗺️ [maps] Searching ({hl}): '{kw}'")
                r = requests.get(
                    api_endpoint,
                    params={
                        "q": kw,
                        "engine": engine,
                        "type": search_type,
                        "api_key": api_key,
                        "hl": hl, # Inject dynamic language
                        "num": max_results_per_keyword,
                    },
                    timeout=request_timeout,
                )

                if r.status_code != 200:
                    print(f" ⚠️ HTTP {r.status_code} for '{kw}'")
                    continue

                payload = r.json()
                if payload.get("error"):
                    print(f" ❌ SerpAPI error '{kw}': {payload['error']}")
                    continue

                results = _get_local_results(payload)
                stats = {"total": len(results), "no_phone": 0, "no_website": 0,

```

```

"low_rating": 0, "low_reviews": 0, "duplicate": 0, "kept": 0}

    for item in results[:max_results_per_keyword]:
        if not isinstance(item, dict): continue

        name = _clean_text(item.get("title") or item.get("name") or
""")
        if not name or len(name) < 2: continue

        phone_raw = _clean_text(item.get("phone") or
item.get("phone_number") or "")
        website = _clean_text(item.get("website") or
item.get("link") or "")
        address = _clean_text(item.get("address") or
item.get("street_address") or "")
        coords = item.get("gps_coordinates") or {}
        location = f"{coords['latitude']}, {coords['longitude']}" if
coords.get("latitude") and coords.get("longitude") else ""
        rating = _to_float(item.get("rating") or 0)
        review_count = _to_int(item.get("reviews") or
item.get("review_count") or 0)
        category = _category_to_str(item.get("type") or
item.get("types") or item.get("category"))

        if require_phone and not phone_raw.strip():
stats["no_phone"] += 1; continue
        if require_website and not website.strip():
stats["no_website"] += 1; continue
        if min_rating > 0 and rating < min_rating:
stats["low_rating"] += 1; continue
        if min_reviews > 0 and review_count < min_reviews:
stats["low_reviews"] += 1; continue

        phone_formatted = _format_phone_e164(phone_raw,
phone_country_prefix)

        if phone_formatted and _phone_is_valid(phone_formatted):
            dedupe_key = f"phone:{phone_formatted}"
        else:
            dedupe_key = f"name:{name.lower().strip()}"

        if dedupe_key in seen: stats["duplicate"] += 1; continue
        seen.add(dedupe_key)

        # Dynamic Intent Score (Generic Global B2B/B2C)
        intent = 20
        if phone_formatted and _phone_is_valid(phone_formatted):
intent += 30

        if website: intent += 20
        if review_count > 10: intent += 15
        if review_count > 50: intent += 10
        if rating >= 4.0: intent += 5

        # Optional Enrichment
        email = _scrape_email_from_website(website) if scrape_emails
else ""

        all_leads.append({
            "name": name,
            "phone": phone_raw,
            "phone_formatted": phone_formatted,
            "email": email,
            "website": website,
            "address": address,

```

```

        "location": location,
        "category": category,
        "source": "maps_business",
        "keyword": kw,
        "rating": rating,
        "review_count": review_count,
        "review_snippet": _clean_text(item.get("description") or
item.get("snippet") or "", 200),
        "destination_visited": "", "review_date": "",
"hotel_reviewed": "",
        "intent_score": intent, "quality_score": "", "segment":
"",
        "last_verified": _today(),
    })
    stats["kept"] += 1

    print(f" 📋 '{kw}' filter stats: {stats}")
    if request_delay_seconds > 0: time.sleep(request_delay_seconds)

except requests.exceptions.Timeout:
    print(f" ⚠️ Timeout: '{kw}'")
except Exception as e:
    print(f" ⚠️ Error '{kw}': {e}")

return all_leads

def _extract_reviewers(
    self, api_key: str, destination: str, max_reviewers: int = 100,
    recency_days: int = 90, min_activity: int = 5,
    api_endpoint: str = SERPAPI_ENDPOINT, request_timeout: int = 30,
    hl: str = "en" # Allow localized reviewer mining
) -> List[Dict[str, Any]]:
    """Extract PEOPLE who recently reviewed hotels/attractions globally."""
    reviewers: List[Dict[str, Any]] = []
    seen_users: set = set()
    place_searches = [
        ("hotels", f"hotels in {destination}"),
        ("attractions", f"tourist attractions in {destination}"),
    ]

    try:
        for label, query in place_searches:
            if len(reviewers) >= max_reviewers: break
            print(f" 👤 [maps_reviewers] Finding {label}: '{destination}'")
            try:
                r = requests.get(
                    api_endpoint,
                    params={"q": query, "engine": "google_maps", "type":
"search", "api_key": api_key, "hl": hl, "num": 10},
                    timeout=request_timeout,
                )
                if r.status_code != 200: continue
                payload = r.json()
            except Exception as e: continue

            if payload.get("error"): continue
            places = _get_local_results(payload)
            place_limit = 6 if label == "hotels" else 3

            for place in places[:place_limit]:
                if len(reviewers) >= max_reviewers: break
                if not isinstance(place, dict): continue

                place_title = place.get("title") or place.get("name") or

```

"Unknown"

```
id_params, id_name = _place_identifier_params(place)
if not id_params: continue

try:
    rr = requests.get(
        api_endpoint,
        params={"engine": "google_maps_reviews", "api_key":
api_key, "sort_by": "newestFirst", "hl": hl, **id_params},
        timeout=request_timeout,
    )
    if rr.status_code != 200: continue
    rev_payload = rr.json()
except Exception: continue
if rev_payload.get("error"): continue

reviews = _get_reviews_from_payload(rev_payload)
stats = {"old": 0, "no_user": 0, "low_activity": 0,
"bad_name": 0, "duplicate": 0, "kept": 0}

for review in reviews:
    if len(reviewers) >= max_reviewers: break
    if not isinstance(review, dict): continue

    review_date = review.get("date", "")
    if not _parse_review_date(review_date, recency_days):
stats["old"] += 1; continue

    user = review.get("user") or review.get("user_info") or
review.get("author") or {}
    if isinstance(user, str): user = {"name": user}
    if not isinstance(user, dict) or not user:
stats["no_user"] += 1; continue

    user_reviews = _get_user_review_count(user)
    if user_reviews < min_activity: stats["low_activity"] +=
1; continue

    name = _clean_text(user.get("name") or
user.get("username") or "")
    if len(name) < 2: stats["bad_name"] += 1; continue

    user_location = _clean_text(user.get("location") or "")
    profile_key = user.get("link") or
user.get("profile_link") or user.get("contributor_id")
    dedupe_key = f"profile:{profile_key}" if profile_key
else f"name:{name.lower()}|loc:{user_location.lower()}"
    if dedupe_key in seen_users: stats["duplicate"] += 1;
continue

    seen_users.add(dedupe_key)

    snippet = review.get("snippet") or review.get("text") or
""

    reviewers.append({
        "name": name, "phone": "", "phone_formatted": "",
"email": "", "website": "",
        "address": _clean_text(place.get("address") or ""),
"location": user_location,
        "category": "Traveler", "source":
"google_maps_reviewer", "keyword": destination,
        "rating": review.get("rating") or 0, "review_count":
user_reviews,
        "review_snippet": _clean_text(snippet, 200),
        "destination_visited": destination, "review_date":
```

```

review_date,
                                "hotel_reviewed": _clean_text(place_title),
                                "intent_score": 70, "quality_score": "", "segment":
"", "last_verified": _today(),
                                })
                                stats["kept"] += 1

                                print(f" 📊 Review stats: {stats}")
except Exception as e:
    print(f" ⚠️ Reviewer mining error '{destination}': {e}")

return reviewers

# -----
# Main entrypoint
# -----

def _run(
    self, topic: str, keywords: List[str], output_dir: str, sources:
List[str] = None,
    credentials_file: str = "", api_endpoint: str = SERPAPI_ENDPOINT,
engine: str = "google_maps",
    search_type: str = "search", hl: str = "en", request_timeout: int = 30,
    max_results_per_keyword: int = 20, skip_if_cached: bool = True,
    require_phone: bool = True, require_website: bool = False, min_rating:
float = 0.0,
    min_reviews: int = 0, phone_country_prefix: str = "+1",
request_delay_seconds: float = 1.0,
    scrape_emails: bool = False, reviewer_recency_days: int = 90,
reviewer_min_activity: int = 5,
    csv_file: str = ""
) -> str:

    # — Sanitize inputs —————
    api_endpoint = str(api_endpoint or SERPAPI_ENDPOINT).strip()
    engine = str(engine or "google_maps").strip()
    search_type = str(search_type or "search").strip()
    hl = str(hl or "en").strip() # Dynamic language
    credentials_file = str(credentials_file or "").strip()
    csv_file = str(csv_file or "").strip()
    sources = [str(s).strip().lower() for s in (sources or ["maps"])]

    if s]
        keywords = [str(kw).strip() for kw in (keywords or []) if kw]
        phone_country_prefix = str(phone_country_prefix or "+1").strip() #
Dynamic prefix
        if not phone_country_prefix.startswith("+"):
            phone_country_prefix = "+" + phone_country_prefix

    # — Output path —————
    out_dir = Path(output_dir) / "raw"
    out_dir.mkdir(parents=True, exist_ok=True)
    out_file = out_dir / "leads_raw.csv"

    if skip_if_cached and _csv_has_data_rows(out_file):
        return f" 🛑 Skipped (cached): {out_file.name}"

    for src in sources:
        if src not in SUPPORTED_SOURCES:
            print(f" ⚠️ Unknown source ignored: '{src}'")

    needs_api = any(s in sources for s in ["maps", "maps_reviewers"])
    api_key = _load_api_key(credentials_file) if needs_api else ""
    if needs_api and not api_key:
        return f" ❌ No api_key found in: '{credentials_file}'."

```

```

mode_label = " + ".join(sources)
print(f"\n 🚀 LeadDataCollect | mode: [{mode_label}] | lang: [{hl}] |
prefix: [{phone_country_prefix}]"
print(f" 🗝️ Keywords ({len(keywords)}): {keywords}")

all_leads: List[Dict[str, Any]] = []

# =====
# SOURCE: csv
# =====
if "csv" in sources and csv_file:
    p = Path(csv_file).expanduser()
    if p.exists():
        try:
            with open(p, newline="", encoding="utf-8") as f:
                reader = csv.DictReader(f)
                count = 0
                for row in reader:
                    if not isinstance(row, dict): continue
                    lead = {field: row.get(field, "") for field in
CSV_FIELDS}

                    if not _clean_text(lead.get("name")): continue
                    if not lead.get("source"): lead["source"] = "csv"
                    if not lead.get("last_verified"):
lead["last_verified"] = _today()
                    if not lead.get("phone_formatted") and
lead.get("phone"):
                        lead["phone_formatted"] =
_format_phone_e164(lead["phone"], phone_country_prefix)
                        all_leads.append(lead)
                        count += 1
                        print(f" 📄 CSV → {count} leads loaded")
                    except Exception as e:
                        print(f" ⚠️ CSV load failed: {e}")

# =====
# SOURCE: maps (Global Business leads)
# =====
if "maps" in sources:
    biz_leads = self._collect_maps_businesses(
        api_key=api_key, keywords=keywords, api_endpoint=api_endpoint,
engine=engine,
        search_type=search_type, hl=hl, request_timeout=request_timeout,
        max_results_per_keyword=max_results_per_keyword,
require_phone=require_phone,
        require_website=require_website, min_rating=min_rating,
min_reviews=min_reviews,
        phone_country_prefix=phone_country_prefix,
request_delay_seconds=request_delay_seconds,
        scrape_emails=scrape_emails
    )
    print(f" ✅ [maps] {len(biz_leads)} business leads")
    all_leads.extend(biz_leads)

# =====
# SOURCE: maps_reviewers (Global Traveler leads)
# =====
if "maps_reviewers" in sources:
    max_per_kw = max(50, max_results_per_keyword * 2)
    total_travelers = 0
    for kw in keywords:
        travelers = self._extract_reviewers(
            api_key=api_key, destination=kw, max_reviewers=max_per_kw,

```

```

recency_days=reviewer_recency_days,
min_activity=reviewer_min_activity,
api_endpoint=api_endpoint, request_timeout=request_timeout,
hl=hl
    )
    print(f"  ✅ [maps_reviewers] '{kw}' → {len(travelers)}
travelers")
    all_leads.extend(travelers)
    total_travelers += len(travelers)
    print(f"  ✅ [maps_reviewers] Total: {total_travelers}")

# =====
# Write output
# =====
if not all_leads:
    _write_leads_csv(out_file, [])
    hint = ""
    if "maps" in sources and require_phone:
        hint = " Tip: set require_phone=false to relax filter."
    return f" ⚠️ No leads collected.{hint}"

_write_leads_csv(out_file, all_leads)

biz_n      = sum(1 for l in all_leads if l.get("source") ==
"maps_business")
trav_n     = sum(1 for l in all_leads if l.get("source") ==
"google_maps_reviewer")
csv_n      = sum(1 for l in all_leads if str(l.get("source","")).lower()
== "csv")
phone_n    = sum(1 for l in all_leads if
_phone_is_valid(l.get("phone_formatted","") or l.get("phone","")))
website_n  = sum(1 for l in all_leads if l.get("website"))

return (
    f"✓ Collected {len(all_leads)} leads → {out_file.name}\n"
    f" 🏢 Businesses (maps):      {biz_n}\n"
    f" 👤 Travelers (reviewers):  {trav_n}\n"
    f" 📄 CSV fallback:           {csv_n}\n"
    f" 📞 With phone:              {phone_n}\n"
    f" 🌐 With website:           {website_n}"
)

```

```

=====
=
"""

```

leaddata_enrich_osint.py – OSINT Enrichment for B2C & B2B Leads

Input:
 {output_dir}/scored/leads_scored.csv (or fallbacks)

Output:
 {output_dir}/enriched/leads_enriched.csv

CSV Column Contract (Critical – never overwrite real with guessed):
 email → REAL only (original source / live-verified / OSINT-found)

possible_email outreach)	→ GUESSED only (pattern permutation, never used for
email_status	→ verified public_found original social_only
guessed none	
email_source	→ where the email came from (URL or method name)
outreach_ready	→ yes no
enriched_email	→ mirrors email (real only, for downstream
compatibility)	
possible_email_conf	→ confidence score for guessed email (0.000-1.000)
enrichment_confidence	→ confidence score for real email (0.000-1.000)
enrichment_method	→ original osint_live_verified social_profile_only
email_guess none	
social_profile_url	→ LinkedIn / Facebook / Instagram direct URL if found
evidence_url	→ source page where real email was discovered

Rule 16: Single output file per tool.

Rule 32: Smart skip if output already exists.

Rule 39: API key resolved via credentials_file JSON {"api_key": "..."}.

```

from pathlib import Path
from typing import Type, List, Optional, Tuple, Dict, Any
import csv
import json
import re
import time
import unicodedata
import requests
from pydantic import BaseModel, Field
from crewai.tools import BaseTool

```

```
SERPAPI_ENDPOINT = "https://serpapi.com/search.json"
```

```
# Names that are pseudonyms, roles, or corporate noise – never valid personal
names
```

```

PSEUDONYMS_AND_JUNK = {
    "guy", "poet", "vegan", "writes", "handyman", "build", "travel", "vacation",
    "guide", "local", "tester", "test", "user", "anonymous", "customer",
    "guest",
    "review", "reviewer", "channel", "vlog", "blog", "photography", "photos",
    "adventures", "adventure", "solutions", "services", "group", "family",
    "dr", "mr", "mrs", "ms", "prof", "doc", "sir", "lady", "jr", "sr",
    "ii", "iii", "iv", "v"
}

```

```

# _____
# Pydantic Input Schema
# _____

```

```

class OSINTEnrichInput(BaseModel):
    input_file: str = Field(..., description="Path to leads_scored.csv")
    output_dir: str = Field(..., description="Root output directory for
enriched/")
    credentials_file: str = Field(default="", description="JSON file with
{api_key: ...}")
    min_confidence: float = Field(default=0.30, description="Min confidence to
keep real email")
    allow_guessing: bool = Field(default=True, description="Generate
possible_email_guesses (never overwrites email)")
    skip_if_cached: bool = Field(default=True, description="Skip if enriched
output already exists")
    max_osint_queries: int = Field(default=50, description="Max live SerpAPI

```



```

search_queries")
    query_delay_seconds: int = Field(default=1, description="Seconds between API
calls")
    max_enrich_rows: int = Field(default=0, description="Limit to top N rows by
intent. 0 = all.")

```

```

# -----
# Utility Helpers
# -----

```

```

def _load_api_key(credentials_file: str) -> str:
    """Load SerpAPI key from JSON credentials file."""
    if not credentials_file:
        return ""
    p = Path(credentials_file).expanduser()
    if not p.exists():
        return ""
    try:
        data = json.loads(p.read_text(encoding="utf-8"))
        return str(data.get("api_key") or "").strip()
    except Exception:
        return ""

```

```

def _strip_accents(text: str) -> str:
    """Normalize accented characters to ASCII (Clément → Clement)."""
    try:
        return unicodedata.normalize("NFKD", text).encode("ascii",
"ignore").decode("ascii")
    except Exception:
        return text

```

```

def _clean_name(name: str) -> str:
    """
    Extract clean First Last from messy reviewer display names.
    Removes brackets, special chars, and noise tokens.
    """
    if not name:
        return ""
    name = re.sub(r"\(.*?\)|\[.*?\]", " ", name)
    cleaned = re.sub(r"^[a-zA-Z\s\u00C0-\u017F'-]", " ", name)
    parts = [p.strip() for p in cleaned.split() if p.strip()]
    noise = {"dr", "mr", "mrs", "ms", "prof", "doc", "jr", "sr", "ii", "iii",
"iv", "v"}
    parts = [p for p in parts if p.lower() not in noise]
    if len(parts) >= 2:
        return f"{parts[0]} {parts[-1]}"
    elif parts:
        return parts[0]
    return ""

```

```

def _is_valid_real_name(first: str, last: str) -> bool:
    """
    Guard: Reject initials, pseudonyms, numbers, and corporate names.
    Must pass before any email guess is generated.
    """
    if len(first) < 2 or len(last) < 2:
        return False
    if any(ch.isdigit() for ch in (first + last)):
        return False
    if first.lower() in PSEUDONYMS_AND_JUNK or last.lower() in

```

```

PSEUDONYMS_AND_JUNK:
    return False
return True

def _extract_city_state(address: str) -> str:
    """
    Parse City, State from a full address string.
    'Vendue Range, Concord St, Charleston, SC 29401' → 'Charleston, SC'
    """
    if not address:
        return ""
    parts = [p.strip() for p in address.split(",") if p.strip()]
    if len(parts) >= 3:
        for part in parts[-3:]:
            if re.search(r'\b[A-Z]{2}\b\s+\d{5}', part):
                return part.strip()
        return f"{parts[-2]}, {parts[-1]}"
    return address

def _extract_emails_from_text(text: str) -> List[str]:
    """Find real email addresses embedded in scraped text."""
    if not text:
        return []
    pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
    found = re.findall(pattern, text)
    return [e.lower().strip() for e in found if "@" in e]

def _csv_has_data_rows(path: Path) -> bool:
    """True only if CSV has at least one non-empty data row after header."""
    if not path.exists() or path.stat().st_size == 0:
        return False
    try:
        with open(path, newline="", encoding="utf-8") as f:
            reader = csv.reader(f)
            next(reader, None)
            for row in reader:
                if any(str(cell).strip() for cell in row):
                    return True
            return False
    except Exception:
        return path.stat().st_size > 0

# -----
# OSINT: Live Web Search for Real Emails
# -----

def _live_osint_social_search(
    name: str,
    city_context: str,
    api_key: str
) -> Tuple[Optional[str], Optional[str], float, Optional[str]]:
    """
    Search Google for real public social profiles and explicitly listed emails.
    """
    if not api_key or len(name) < 3:
        return None, None, 0.0, None

    query = (
        f'"{name}" "{city_context}" '
        f'(site:linkedin.com/in/ OR site:facebook.com/ OR site:instagram.com/) '

```

```

        f'"gmail.com"'
    )

    try:
        response = requests.get(
            SERPAPI_ENDPOINT,
            params={
                "q": query,
                "engine": "google",
                "hl": "en",
                "gl": "us",
                "api_key": api_key,
                "num": 5,
            },
            timeout=25
        )

        if response.status_code != 200:
            return None, None, 0.0, None

        organic = response.json().get("organic_results", [])

        for result in organic:
            link = result.get("link", "")
            snippet = result.get("snippet", "")
            title = result.get("title", "")

            # Priority 1: Real email found in indexed snippet
            emails = _extract_emails_from_text(f"{title} {snippet}")
            is_social = any(p in link for p in ["linkedin.com/in/",
"facebook.com", "instagram.com"])

            if emails:
                social_url = link if is_social else None
                return emails[0], social_url, 0.90, link # ✅ Real verified
email

            # Priority 2: Social profile found but no email
            if is_social:
                return None, link, 0.60, link # 🔗 Profile only

    except Exception:
        pass

    return None, None, 0.0, None

def _live_osint_google_search(
    name: str,
    location: str,
    api_key: str
) -> Tuple[List[str], float, Optional[str]]:
    """
    Broader Google search for emails tied to person + location.
    Fallback after social search finds nothing.
    """
    if not api_key or len(name) < 3:
        return [], 0.0, None

    queries = [
        f'"{name}" "{location}" "gmail.com"',
        f'"{name}" "contact" "{location}"',
    ]

```

```

found_emails: List[str] = []
best_confidence = 0.0
evidence_url: Optional[str] = None

for idx, query in enumerate(queries):
    try:
        if idx > 0:
            time.sleep(1.0)

        response = requests.get(
            SERPAPI_ENDPOINT,
            params={
                "q": query,
                "engine": "google",
                "hl": "en",
                "gl": "us",
                "api_key": api_key,
                "num": 10,
            },
            timeout=25
        )

        if response.status_code != 200:
            continue

        for result in response.json().get("organic_results", []):
            text = f"{result.get('title', '')} {result.get('snippet', '')}"
            link = result.get("link", "")
            emails = _extract_emails_from_text(text)

            for email in emails:
                if email not in found_emails:
                    found_emails.append(email)
                    best_confidence = max(best_confidence, 0.85)
                    if not evidence_url:
                        evidence_url = link

    except Exception:
        continue

return found_emails, best_confidence, evidence_url

```

```

# -----
# Gussed Emails (possible_email only)
# -----

```

```

def _guess_consumer_emails(first_name: str, last_name: str) -> List[Tuple[str,
float]]:
    """
    Generate plausible email permutations for the possible_email column ONLY.
    These are NEVER written to the email column.
    """
    if not _is_valid_real_name(first_name, last_name):
        return []

    fn = _strip_accents(first_name.lower().replace("-", "").replace(".",
""))
    ln = _strip_accents(last_name.lower().replace("-", "").replace(".",
""))

    if not fn or not ln:
        return []

```

```

domains = [
    ("gmail.com", 0.55),
    ("yahoo.com", 0.25),
    ("outlook.com", 0.15),
    ("hotmail.com", 0.10),
]

patterns = [
    (f"{fn}.{ln}", 0.80), # john.doe
    (f"{fn}{ln}", 0.50), # johndoe
    (f"{fn[0]}{ln}", 0.35), # jdoe
    (f"{fn}.{ln[0]}", 0.25), # john.d
]

results = []
for domain, dom_conf in domains:
    for pattern, pat_conf in patterns:
        email = f"{pattern}@{domain}"
        # Guesses are heavily penalized to reflect uncertainty
        combined_conf = round((pat_conf * dom_conf) * 0.1, 3)
        results.append((email, combined_conf))

return sorted(results, key=lambda x: x[1], reverse=True)

# -----
# Main Tool Class
# -----

class OSINTEnrichTool(BaseTool):
    name: str = "leaddata_enrich_osint"
    description: str = (
        "Enrich B2C traveler leads with REAL verified emails (OSINT) "
        "and plausible guessed emails (possible_email). "
        "Real emails go to 'email'. Guesses go to 'possible_email'. "
        "Never mixed. outreach_ready=yes only for verified contacts."
    )
    args_schema: Type[BaseModel] = OSINTEnrichInput

    def _run(
        self,
        input_file: str,
        output_dir: str,
        credentials_file: str = "",
        min_confidence: float = 0.30,
        allow_guessing: bool = True,
        skip_if_cached: bool = True,
        max_osint_queries: int = 50,
        query_delay_seconds: int = 1,
        max_enrich_rows: int = 0,
    ) -> str:

        # — Input validation -----
        if not input_file or str(input_file).strip().lower() in ("none", "null",
        "")):
            return "❌ Missing valid input_file path"

        # — Output setup -----
        out_dir = Path(output_dir) / "enriched"
        out_dir.mkdir(parents=True, exist_ok=True)
        out_file = out_dir / "leads_enriched.csv"

        # Rule 32: Smart skip
        if skip_if_cached and _csv_has_data_rows(out_file):

```

```

        return f"🔍 Skipped (cached): {out_file.name}"

# — Input file resolution with fallback chain —————
in_path = Path(input_file).expanduser()
if not in_path.exists():
    parent = in_path.parent.parent
    fallbacks = [
        parent / "scored"      / "leads_scored.csv",
        parent / "normalized" / "leads_clean.csv",
        parent / "raw"         / "leads_raw.csv",
    ]
    for fb in fallbacks:
        if fb.exists():
            in_path = fb
            print(f"📁 Input not found – using fallback: {fb.name}")
            break

if not in_path.exists():
    return f"❌ Input not found: {input_file} (checked standard
fallbacks)"

# — Load API key —————
api_key = _load_api_key(credentials_file)
if not api_key:
    print(f"⚠️ No SerpAPI key – OSINT searches skipped. Guesses only
(if allow_guessing=True).")

# — Read CSV —————
print(f"📂 Reading: {in_path.name}")
try:
    with open(in_path, "r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        src_fields: List[str] = list(reader.fieldnames) if
reader.fieldnames else []
        rows = list(reader)
    except Exception as e:
        return f"❌ Failed to parse CSV: {e}"

total_input = len(rows)

# — Sort by intent score, limit if requested —————
try:
    rows.sort(key=lambda r: float(r.get("intent_score", 0) or 0),
reverse=True)
except Exception:
    pass

if max_enrich_rows > 0 and len(rows) > max_enrich_rows:
    print(f"🎯 Limiting to top {max_enrich_rows} / {total_input} rows
by intent score")
    rows = rows[:max_enrich_rows]

# — Extend CSV schema with new columns —————
NEW_COLUMNS = [
    "email", # REAL only – verified/found
    "possible_email", # GUESSED only – pattern permutation
    "possible_email_conf", # Confidence for guessed email
    "email_status", # verified|public_found|original|
social_only|guessed|none
    "email_source", # method name or URL where real email
was found
    "evidence_url", # page where real email was discovered
    "outreach_ready", # yes | no
    "enriched_email", # mirrors email (backward compatibility)

```

```

        "enrichment_confidence",    # confidence for real email
        "enrichment_method",       # osint_live_verified|
social_profile_only|email_guess|original|none
        "social_profile_url",      # LinkedIn / Facebook / Instagram URL
        "enriched_phone",         # phone (kept for downstream
compatibility)
    ]
    for col in NEW_COLUMNS:
        if col not in src_fields:
            src_fields.append(col)

# — Stats counters —————
stats = {
    "total": len(rows),
    "original_email": 0,
    "osint_real_email": 0,
    "social_profile_only": 0,
    "guessed_only": 0,
    "no_contact": 0,
}

enriched_rows = []
osint_queries_run = 0

print(f" ⚡ Enriching {len(rows)} leads...")
print(f" 🔑 OSINT budget: {max_osint_queries} queries | Guessing:
{allow_guessing}")
print(f" 📊 Dual-track: email=REAL | possible_email=GUESSED")

for idx, row in enumerate(rows, 1):
    name = row.get("name", "").strip()
    location = row.get("location", "").strip()
    address = row.get("address", "").strip()
    source = row.get("source", "")

# — Initialize all output fields as empty —————
real_email: Optional[str] = None
guessed_email: Optional[str] = None
guessed_conf: float = 0.0
real_confidence: float = 0.0
method: str = "none"
email_status: str = "none"
email_source: str = ""
evidence_url: Optional[str] = None
social_url: Optional[str] = None
outreach_ready: str = "no"

# —————
# CASE 1: Already has original email → keep as-is
# —————
existing_email = row.get("email", "").strip()
if existing_email and "@" in existing_email:
    real_email = existing_email
    real_confidence = 1.0
    method = "original"
    email_status = "original"
    email_source = "source_data"
    outreach_ready = "yes"
    stats["original_email"] += 1
    print(f" ✅ [{idx}/{len(rows)}] Original email kept:
{real_email}")

# —————
# CASE 2: B2C Traveler → OSINT social + web search

```

```

# -----
elif source == "google_maps_reviewer" and len(name) >= 3:
    parsed_name = _clean_name(name)
    parts = parsed_name.split()
    first_name = parts[0] if parts else ""
    last_name = parts[-1] if len(parts) > 1 else ""

    # Skip names with no Latin characters
    if not re.search(r'[a-zA-Z]', name):
        first_name = ""
        last_name = ""

    # Determine best city context for search accuracy
    city_context = _extract_city_state(address) or location or ""

    # — Method A: Live Social OSINT (primary) —
    if api_key and osint_queries_run < max_osint_queries and
parsed_name:
        print(f" 🔍 [{idx}/{len(rows)}] Social OSINT:
{parsed_name} | {city_context}")

        found_email, found_social, conf, ev_url =
_live_osint_social_search(
            parsed_name, city_context, api_key
        )
        osint_queries_run += 1

        if found_email and conf >= min_confidence:
            real_email = found_email
            real_confidence = conf
            method = "osint_live_verified"
            email_status = "verified"
            email_source = ev_url or "social_osint"
            evidence_url = ev_url
            outreach_ready = "yes"
            stats["osint_real_email"] += 1
            print(f" 📧 REAL Email found: {real_email}
(conf={conf:.2f})")

            if found_social:
                social_url = found_social
                if not real_email:
                    method = "social_profile_only"
                    email_status = "social_only"
                    evidence_url = found_social
                    real_confidence = conf
                    stats["social_profile_only"] += 1
                    print(f" 🔗 Social Profile: {social_url}")

        # — Method B: Broader Google search (secondary) —
        if not real_email and api_key and osint_queries_run <
max_osint_queries:
            live_emails, live_conf, live_ev =
_live_osint_google_search(
                parsed_name, city_context, api_key
            )
            osint_queries_run += 1

            if live_emails and live_conf >= min_confidence:
                real_email = live_emails[0]
                real_confidence = live_conf
                method = "osint_live_verified"
                email_status = "public_found"
                email_source = live_ev or "google_osint"

```



```

        evidence_url    = live_ev
        outreach_ready  = "yes"
        stats["osint_real_email"] += 1
        print(f"🔗 REAL Email (web): {real_email}

(conf={live_conf:.2f})")

        if query_delay_seconds > 0:
            time.sleep(query_delay_seconds)

        # — Method C: Pattern guessing (possible_email only) —
        if allow_guessing and first_name and last_name:
            guesses = _guess_consumer_emails(first_name, last_name)
            if guesses:
                guessed_email, guessed_conf = guesses[0]
                if not real_email:
                    method          = "email_guess"
                    email_status    = "guessed"
                    outreach_ready  = "no"    # guesses NEVER mark
                    stats["guessed_only"] += 1
                    print(f"🔗 Guessed: {guessed_email}

(conf={guessed_conf:.3f})")

                if not real_email and not guessed_email and not social_url:
                    stats["no_contact"] += 1

# -----
# CASE 3: B2B business lead – no OSINT enrichment needed
# -----
else:
    existing_website = row.get("website", "").strip()
    if existing_email:
        real_email      = existing_email
        real_confidence = 1.0
        method          = "original"
        email_status    = "original"
        outreach_ready  = "yes"
        stats["original_email"] += 1
    elif existing_website:
        email_status    = "none"
        outreach_ready  = "no"
    else:
        stats["no_contact"] += 1

# -----
# WRITE ALL COLUMNS – strict separation maintained
# -----

# Real email – only verified/found/original
row["email"] = real_email or ""

# Guessed email – NEVER touches email column
row["possible_email"] = guessed_email or ""
row["possible_email_conf"] = f"{guessed_conf:.3f}" if
guessed_email else ""

# Metadata
row["email_status"] = email_status
row["email_source"] = email_source
row["evidence_url"] = evidence_url or ""
row["outreach_ready"] = outreach_ready

# Downstream compatibility columns
row["enriched_email"] = real_email or ""

```

```

        row["enrichment_confidence"] = f"{real_confidence:.3f}"
        row["enrichment_method"]      = method
        row["social_profile_url"]      = social_url or ""
        row["enriched_phone"]         = row.get("phone", "")

    enriched_rows.append(row)

# — Write output CSV —————
try:
    with open(out_file, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=src_fields,
extrasaction="ignore")
        w.writeheader()
        w.writerows(enriched_rows)
except Exception as e:
    return f"❌ Failed to write output: {e}"

# — Build summary —————
real_count      = stats["original_email"] + stats["osint_real_email"]
guessed_count   = stats["guessed_only"]
social_count    = stats["social_profile_only"]
no_count       = stats["no_contact"]

return (
    f"✓ Enrichment complete → {out_file.name}\n"
    f" 📊 Total processed      : {stats['total']}\n"
    f" 🎯 Real emails found    : {real_count} "
    f"({stats['original_email']} original + {stats['osint_real_email']}
OSINT-verified)\n"
    f" 🔗 Social profiles     : {social_count} (no email, but direct
message URL)\n"
    f" 💭 Guessed emails      : {guessed_count} (possible_email column
only, outreach_ready=no)\n"
    f" ⚠️ No contact found    : {no_count}\n"
    f" 🔑 OSINT queries used: {osint_queries_run}/{max_osint_queries}\n"
    f"
    f" 📌 Columns: email=REAL | possible_email=GUESSED |
outreach_ready=yes/no"
)

```

=====

=
 """"

leaddata_export.py – Export & Stats Tool (Rule 16: single output)

Reads: {output_dir}/scored/leads_scored.csv

Writes:

- {output_dir}/scored/leads_scored.json (if json in formats)
 - {output_dir}/insights/{stats_file} (if generate_stats)
 """"

```

from pathlib import Path
from typing import Type, List
import csv
import json
from collections import defaultdict
from pydantic import BaseModel, Field
from crewai.tools import BaseTool

```

```

class LeadDataExportInput(BaseModel):
    output_dir: str = Field(...)
    formats: List[str] = Field(default=["csv", "json"])
    generate_stats: bool = Field(default=True)
    stats_file: str = Field(default="lead_stats.json")

```

```
include_segments_breakdown: bool = Field(default=True)
```

```
class LeadDataExportTool(BaseTool):
    name: str = "leaddata_export"
    description: str = "Export final leads + generate stats."
    args_schema: Type[BaseModel] = LeadDataExportInput

    def _run(self, output_dir: str,
              formats: List[str] = None,
              generate_stats: bool = True,
              stats_file: str = "lead_stats.json",
              include_segments_breakdown: bool = True) -> str:

        if formats is None:
            formats = ["csv", "json"]

        scored_csv = Path(output_dir) / "scored" / "leads_scored.csv"
        if not scored_csv.exists():
            return f"✗ Input missing: {scored_csv}"

        with open(scored_csv, 'r', encoding='utf-8') as f:
            records = list(csv.DictReader(f))

        # JSON export
        if "json" in formats:
            json_file = Path(output_dir) / "scored" / "leads_scored.json"
            with open(json_file, 'w', encoding='utf-8') as f:
                json.dump(records, f, indent=2, ensure_ascii=False)

        # Stats
        if not generate_stats:
            return f"✓ Exported {len(records)} (stats skipped)"

        segments = defaultdict(int)
        scores = []
        for r in records:
            segments[r.get("segment", "unscored")] += 1
            try:
                scores.append(int(r.get("quality_score", 0)))
            except (ValueError, TypeError):
                pass

        stats = {
            "total": len(records),
            "with_phone": sum(1 for r in records if r.get("phone")),
            "with_email": sum(1 for r in records if r.get("email")),
            "with_website": sum(1 for r in records if r.get("website")),
            "avg_quality_score": round(sum(scores) / len(scores), 1) if scores
        }
    else 0,
    }
    if include_segments_breakdown:
        stats["segments"] = dict(segments)

    insights_dir = Path(output_dir) / "insights"
    insights_dir.mkdir(parents=True, exist_ok=True)
    stats_path = insights_dir / stats_file
    with open(stats_path, 'w', encoding='utf-8') as f:
        json.dump(stats, f, indent=2)

    return (f"✓ Exported {len(records)} | "
            f"avg={stats['avg_quality_score']} → {stats_path.name}")
```

```

=====
=
"""
leaddata_geo_resolver.py – Dynamic Global Context Resolver Utility
"""
from typing import Dict

GLOBAL_REGION_MAP = {
    "usa": {"prefix": "+1", "code": "US", "hl": "en"},
    "united states": {"prefix": "+1", "code": "US", "hl": "en"},
    "canada": {"prefix": "+1", "code": "CA", "hl": "en"},
    "ontario": {"prefix": "+1", "code": "CA", "hl": "en"}, # Added for your
specific topic
    "mexico": {"prefix": "+52", "code": "MX", "hl": "es"},
    "uk": {"prefix": "+44", "code": "GB", "hl": "en"},
    "united kingdom": {"prefix": "+44", "code": "GB", "hl": "en"},
    "france": {"prefix": "+33", "code": "FR", "hl": "fr"},
    "germany": {"prefix": "+49", "code": "DE", "hl": "de"},
    "australia": {"prefix": "+61", "code": "AU", "hl": "en"},
    "india": {"prefix": "+91", "code": "IN", "hl": "en"},
}

def resolve_global_context(topic: str, config: dict) -> Dict[str, str]:
    topic_lower = topic.lower()
    for key, meta in GLOBAL_REGION_MAP.items():
        if key in topic_lower:
            return meta

    explicit_prefix = config.get("collect_config", {}).get("maps_config",
{}).get("phone_country_prefix")
    explicit_code = config.get("normalize_config",
{}).get("phone_country_default")
    explicit_hl = config.get("collect_config", {}).get("hl")

    if explicit_prefix or explicit_code:
        return {"prefix": explicit_prefix or "+1", "code": explicit_code or
"US", "hl": explicit_hl or "en"}

    return {"prefix": "+1", "code": "US", "hl": "en"}

=====
=
"""
leaddata_intent.py – Dynamic Topic-Oriented Intent Detector (CF2 Tool)

Replaces the hardcoded travel intent tool. This version dynamically searches
for ANY topic (equipment, SaaS, real estate, travel, etc.) to find people
exhibiting high buying/research intent on forums, Reddit, and Q&A sites.

Input: topic (e.g., "Commercial Truck Leasing"), keywords
Output: {output_dir}/raw/leads_raw.csv (Appends to existing pipeline)

Rule 16: Single output file per tool (appends to raw).
Rule 32: Smart Skip mandatory.
Rule 39: API key resolved via credentials_file.
"""

import logging
from pathlib import Path
from typing import Type, List, Dict, Any, Set
import csv
import json
import re
import time

```

```

import requests
from datetime import datetime
from pydantic import BaseModel, Field
from crewai.tools import BaseTool

# Initialize standard CF2 logger
logger = logging.getLogger(__name__)

SERPAPI_ENDPOINT = "https://serpapi.com/search.json"

# Universal buyer intent modifiers (applies to ANY industry/topic)
INTENT_PHRASES = {
    "high": [
        "looking for", "need recommendations", "how much does", "pricing for",
        "looking to buy", "best company for", "quotes for", "who is the best",
        "need a loan for", "financing options for", "supplier for", "vendor for"
    ],
    "medium": [
        "vs", "compared to", "alternatives to", "reviews of", "is it worth",
        "pros and cons", "experience with", "thinking about buying", "thinking
about switching"
    ],
    "low": [
        "what is", "how to", "guide to", "basics of", "meaning of"
    ]
}

CSV_FIELDS = [
    "name", "phone", "phone_formatted", "email", "website",
    "address", "location", "category", "source", "keyword",
    "rating", "review_count", "review_snippet",
    "destination_visited", "review_date", "hotel_reviewed",
    "intent_score", "quality_score", "segment", "last_verified",
]

class IntentSearchInput(BaseModel):
    topic: str = Field(...)
    keywords: List[str] = Field(...)
    output_dir: str = Field(...)
    credentials_file: str = Field(default="")
    max_results_per_keyword: int = Field(default=10)
    skip_if_cached: bool = Field(default=True)

def _load_api_key(credentials_file: str) -> str:
    """Load SerpAPI key from JSON credentials file (CF2 Standard Format)."""
    if not credentials_file: return ""
    p = Path(credentials_file).expanduser()
    if not p.exists(): return ""
    try:
        data = json.loads(p.read_text(encoding="utf-8"))
        # Flat format check
        key = str(data.get("api_key") or "").strip()
        if key: return key
        # CF2 Multi-key array format (matches your exact JSON structure)
        for k in data.get("keys", []):
            if k.get("status") == "active" and k.get("api_key"):
                return str(k["api_key"]).strip()
        return ""
    except Exception:
        return ""

```

```

def _extract_author(text: str, url: str) -> str:
    reddit_match = re.search(r'reddit\.com/user/([^\s/]+)/', url)
    if reddit_match: return reddit_match.group(1)
    quora_match = re.search(r'quora\.com/profile/([^\s/]+)/', url)
    if quora_match: return quora_match.group(1).replace("-", " ").title()
    asked_match = re.search(r'(?:(?:asked by|posted by|author:)\s*([A-Za-z0-9_]+))',
text, re.IGNORECASE)
    if asked_match: return asked_match.group(1)
    return "Forum Poster"

def _calculate_intent_score(text: str) -> int:
    if not text: return 0
    text_lower = text.lower()
    score = 0

    for phrase in INTENT_PHRASES["high"]:
        if phrase in text_lower: score += 40; break
    for phrase in INTENT_PHRASES["medium"]:
        if phrase in text_lower: score += 25; break
    for phrase in INTENT_PHRASES["low"]:
        if phrase in text_lower: score += 10; break

    if "?" in text: score += 10
    return min(score, 100)

def _get_existing_urls(path: Path) -> Set[str]:
    urls = set()
    if not path.exists(): return urls
    try:
        with open(path, 'r', encoding='utf-8') as f:
            for row in csv.DictReader(f):
                if row.get("source") == "intent_osint" and row.get("website"):
                    urls.add(row["website"])
    except Exception: pass
    return urls

class LeadDataIntentTool(BaseTool):
    name: str = "leaddata_intent_detector"
    description: str = (
        "Dynamically search forums, Reddit, and Q&A sites for people showing "
        "buying/research intent for ANY given topic and keywords."
    )
    args_schema: Type[BaseModel] = IntentSearchInput

    def _run(
        self,
        topic: str,
        keywords: List[str],
        output_dir: str,
        credentials_file: str = "",
        max_results_per_keyword: int = 10,
        skip_if_cached: bool = True,
        **kwargs # Catches force_reddit, force_quora, etc. from YAML
    ) -> str:

        out_dir = Path(output_dir) / "raw"
        out_dir.mkdir(parents=True, exist_ok=True)
        out_file = out_dir / "leads_raw.csv"

        api_key = _load_api_key(credentials_file)
        if not api_key:

```

```

key.")
    logger.error("No SerpAPI key found. Intent search requires an API
    return "❌ No SerpAPI key found. Intent search requires an API key."

existing_urls = _get_existing_urls(out_file)
new_leads = []
total_queries = 0

logger.info(f"🔍 Searching for Intent Topic: '{topic}')"

for kw in keywords:
    if total_queries >= 50:
        logger.warning("API query limit (50) reached.")
        break

    # — DYNAMIC QUERY BUILDER BASED ON YAML FLAGS —————
    force_reddit = kwargs.get("force_reddit", False)
    force_quora = kwargs.get("force_quora", False)

    if force_reddit:
        query = f'"{kw}" (site:reddit.com/r/smallbusiness OR
site:reddit.com/r/Entrepreneur OR site:reddit.com/r/trucking OR
site:reddit.com/r/construction)'
    elif force_quora:
        query = f'"{kw}" (site:quora.com)'
    else:
        query = f'"{kw}" (site:reddit.com OR site:quora.com OR "forum"
OR "thread")'

    try:
        logger.info(f"🔍 Querying: {kw[:60]}...")
        response = requests.get(
            SERPAPI_ENDPOINT,
            params={
                "q": query,
                "engine": "google",
                "api_key": api_key,
                "num": max_results_per_keyword,
                "tbs": "qdr:m"
            },
            timeout=30
        )
        total_queries += 1

        if response.status_code != 200:
            logger.warning(f"HTTP {response.status_code} for keyword:
{kw}")
            continue

        results = response.json().get("organic_results", [])
        if not results:
            logger.info(f"No results found for: {kw}")

        for result in results:
            link = result.get("link", "")
            if link in existing_urls: continue

            snippet = result.get("snippet", "")
            title = result.get("title", "")
            combined_text = f"{title} {snippet}"

            intent_score = _calculate_intent_score(combined_text)
            #if intent_score < 25: continue
            if intent_score < 15: continue # Catches more "warm"

```

research-phase leads

```
        author = _extract_author(combined_text, link)

        new_leads.append({
            "name": author,
            "phone": "",
            "phone_formatted": "",
            "email": "",
            "website": link,
            "address": "",
            "location": "",
            "category": topic,
            "source": "intent_osint",
            "keyword": kw,
            "rating": 0,
            "review_count": 0,
            "review_snippet": snippet[:200],
            "destination_visited": "",
            "review_date": datetime.now().strftime("%Y-%m-%d"),
            "hotel_reviewed": "",
            "intent_score": intent_score,
            "quality_score": "",
            "segment": "",
            "last_verified": datetime.now().strftime("%Y-%m-%d"),
        })
        existing_urls.add(link)
        logger.debug(f"Found high-intent lead: {author} (Score:
{intent_score})")

        time.sleep(1.0)

    except requests.exceptions.Timeout:
        logger.error(f"Timeout while searching for: {kw}")
    except Exception as e:
        logger.error(f"Error searching '{kw}': {e}")
        continue

if not new_leads:
    logger.warning(f"No high-intent leads found for topic: {topic}")
    return f"⚠️ No high-intent leads found for topic: {topic}"

try:
    write_header = not out_file.exists() or out_file.stat().st_size == 0
    with open(out_file, 'a', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=CSV_FIELDS,
extrasaction='ignore')
        if write_header: w.writeheader()
        w.writerows(new_leads)
    except Exception as e:
        logger.error(f"Failed to write to CSV: {e}")
        return f"❌ Failed to write leads to CSV."

    avg_intent = sum(l["intent_score"] for l in new_leads) / len(new_leads)
    success_msg = (
        f"✓ Added {len(new_leads)} intent leads for '{topic}' →
raw/leads_raw.csv | "
        f"Avg Intent Score: {avg_intent:.1f}/100"
    )
    logger.info(success_msg)
    return success_msg
```

=====

=


```
"""
```

```
leaddata_intent.py – Dynamic Topic-Oriented Intent Detector (CF2 Tool)
```

Replaces the hardcoded travel intent tool. This version dynamically searches for ANY topic (equipment, SaaS, real estate, travel, etc.) to find people exhibiting high buying/research intent on forums, Reddit, and Q&A sites.

Input: topic (e.g., "Commercial Truck Leasing"), keywords

Output: {output_dir}/raw/leads_raw.csv (Appends to existing pipeline)

Rule 16: Single output file per tool (appends to raw).

Rule 32: Smart Skip mandatory.

Rule 39: API key resolved via credentials_file.

```
"""
```

```
import logging
from pathlib import Path
from typing import Type, List, Dict, Any, Set
import csv
import json
import re
import time
import requests
from datetime import datetime
from pydantic import BaseModel, Field
from crewai.tools import BaseTool

# Initialize standard CF2 logger
logger = logging.getLogger(__name__)

SERPAPI_ENDPOINT = "https://serpapi.com/search.json"

# Universal buyer intent modifiers (applies to ANY industry/topic)
INTENT_PHRASES = {
    "high": [
        "looking for", "need recommendations", "how much does", "pricing for",
        "looking to buy", "best company for", "quotes for", "who is the best",
        "need a loan for", "financing options for", "supplier for", "vendor for"
    ],
    "medium": [
        "vs", "compared to", "alternatives to", "reviews of", "is it worth",
        "pros and cons", "experience with", "thinking about buying", "thinking
about switching"
    ],
    "low": [
        "what is", "how to", "guide to", "basics of", "meaning of"
    ]
}

CSV_FIELDS = [
    "name", "phone", "phone_formatted", "email", "website",
    "address", "location", "category", "source", "keyword",
    "rating", "review_count", "review_snippet",
    "destination_visited", "review_date", "hotel_reviewed",
    "intent_score", "quality_score", "segment", "last_verified",
]

class IntentSearchInput(BaseModel):
    topic: str = Field(...)
    keywords: List[str] = Field(...)
    output_dir: str = Field(...)
    credentials_file: str = Field(default="")
    max_results_per_keyword: int = Field(default=10)
```

```

skip_if_cached: bool = Field(default=True)

def _load_api_key(credentials_file: str) -> str:
    """Load SerpAPI key from JSON credentials file (CF2 Standard Format)."""
    if not credentials_file: return ""
    p = Path(credentials_file).expanduser()
    if not p.exists(): return ""
    try:
        data = json.loads(p.read_text(encoding="utf-8"))
        # Flat format check
        key = str(data.get("api_key") or "").strip()
        if key: return key
        # CF2 Multi-key array format (matches your exact JSON structure)
        for k in data.get("keys", []):
            if k.get("status") == "active" and k.get("api_key"):
                return str(k["api_key"]).strip()
        return ""
    except Exception:
        return ""

def _extract_author(text: str, url: str) -> str:
    reddit_match = re.search(r'reddit\.com/user/([^\s/]+)/', url)
    if reddit_match: return reddit_match.group(1)
    quora_match = re.search(r'quora\.com/profile/([^\s/]+)/', url)
    if quora_match: return quora_match.group(1).replace("-", " ").title()
    asked_match = re.search(r'(?:(?:asked by|posted by|author:))\s*([A-Za-z0-9_]+)',
text, re.IGNORECASE)
    if asked_match: return asked_match.group(1)
    return "Forum Poster"

def _calculate_intent_score(text: str) -> int:
    if not text: return 0
    text_lower = text.lower()
    score = 0

    for phrase in INTENT_PHRASES["high"]:
        if phrase in text_lower: score += 40; break
    for phrase in INTENT_PHRASES["medium"]:
        if phrase in text_lower: score += 25; break
    for phrase in INTENT_PHRASES["low"]:
        if phrase in text_lower: score += 10; break

    if "?" in text: score += 10
    return min(score, 100)

def _get_existing_urls(path: Path) -> Set[str]:
    urls = set()
    if not path.exists(): return urls
    try:
        with open(path, 'r', encoding='utf-8') as f:
            for row in csv.DictReader(f):
                if row.get("source") == "intent_osint" and row.get("website"):
                    urls.add(row["website"])
    except Exception: pass
    return urls

class LeadDataIntentTool(BaseTool):
    name: str = "leaddata_intent_detector"
    description: str = (

```

```

        "Dynamically search forums, Reddit, and Q&A sites for people showing "
        "buying/research intent for ANY given topic and keywords."
    )
args_schema: Type[BaseModel] = IntentSearchInput

def _run(
    self,
    topic: str,
    keywords: List[str],
    output_dir: str,
    credentials_file: str = "",
    max_results_per_keyword: int = 10,
    skip_if_cached: bool = True,
    **kwargs # Catches force_reddit, force_quora, etc. from YAML
) -> str:

    out_dir = Path(output_dir) / "raw"
    out_dir.mkdir(parents=True, exist_ok=True)
    out_file = out_dir / "leads_raw.csv"

    api_key = _load_api_key(credentials_file)
    if not api_key:
        logger.error("No SerpAPI key found. Intent search requires an API
key.")
        return "❌ No SerpAPI key found. Intent search requires an API key."

    existing_urls = _get_existing_urls(out_file)
    new_leads = []
    total_queries = 0

    logger.info(f"🔍 Searching for Intent Topic: '{topic}'")

    for kw in keywords:
        if total_queries >= 50:
            logger.warning("API query limit (50) reached.")
            break

        # — DYNAMIC QUERY BUILDER BASED ON YAML FLAGS —————
        force_reddit = kwargs.get("force_reddit", False)
        force_quora = kwargs.get("force_quora", False)

        if force_reddit:
            query = f'"{kw}" (site:reddit.com/r/smallbusiness OR
site:reddit.com/r/Entrepreneur OR site:reddit.com/r/trucking OR
site:reddit.com/r/construction)'
        elif force_quora:
            query = f'"{kw}" (site:quora.com)'
        else:
            query = f'"{kw}" (site:reddit.com OR site:quora.com OR "forum"
OR "thread")'

        try:
            logger.info(f"🔍 Querying: {kw[:60]}...")
            response = requests.get(
                SERPAPI_ENDPOINT,
                params={
                    "q": query,
                    "engine": "google",
                    "api_key": api_key,
                    "num": max_results_per_keyword,
                    "tbs": "qdr:m"
                },
                timeout=30
            )

```

```

total_queries += 1

if response.status_code != 200:
    logger.warning(f"HTTP {response.status_code} for keyword:
{kw}")
    continue

results = response.json().get("organic_results", [])
if not results:
    logger.info(f"No results found for: {kw}")

for result in results:
    link = result.get("link", "")
    if link in existing_urls: continue

    snippet = result.get("snippet", "")
    title = result.get("title", "")
    combined_text = f"{title} {snippet}"

    intent_score = _calculate_intent_score(combined_text)
    #if intent_score < 25: continue
    if intent_score < 15: continue # Catches more "warm"
research-phase leads

    author = _extract_author(combined_text, link)

    new_leads.append({
        "name": author,
        "phone": "",
        "phone_formatted": "",
        "email": "",
        "website": link,
        "address": "",
        "location": "",
        "category": topic,
        "source": "intent_osint",
        "keyword": kw,
        "rating": 0,
        "review_count": 0,
        "review_snippet": snippet[:200],
        "destination_visited": "",
        "review_date": datetime.now().strftime("%Y-%m-%d"),
        "hotel_reviewed": "",
        "intent_score": intent_score,
        "quality_score": "",
        "segment": "",
        "last_verified": datetime.now().strftime("%Y-%m-%d"),
    })
    existing_urls.add(link)
    logger.debug(f"Found high-intent lead: {author} (Score:
{intent_score})")

    time.sleep(1.0)

except requests.exceptions.Timeout:
    logger.error(f"Timeout while searching for: {kw}")
except Exception as e:
    logger.error(f"Error searching '{kw}': {e}")
    continue

if not new_leads:
    logger.warning(f"No high-intent leads found for topic: {topic}")
    return f"⚠️ No high-intent leads found for topic: {topic}"

```

```

try:
    write_header = not out_file.exists() or out_file.stat().st_size == 0
    with open(out_file, 'a', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=CSV_FIELDS,
extrasaction='ignore')
        if write_header: w.writeheader()
        w.writerows(new_leads)
except Exception as e:
    logger.error(f"Failed to write to CSV: {e}")
    return f"❌ Failed to write leads to CSV."

avg_intent = sum(l["intent_score"] for l in new_leads) / len(new_leads)
success_msg = (
    f"✓ Added {len(new_leads)} intent leads for '{topic}' →
raw/leads_raw.csv | "
    f"Avg Intent Score: {avg_intent:.1f}/100"
)
logger.info(success_msg)
return success_msg

```

```

=====
=
"""
leaddata_linkedin.py – LinkedIn Company Directory Scraper (CF2 Tool)
Source: Google Search targeting site:linkedin.com/company
Output: {output_dir}/raw/leads_raw.csv
"""

import logging
import csv
import json
import time
import re
import requests
from pathlib import Path
from typing import Type, List
from pydantic import BaseModel, Field
from crewai.tools import BaseTool

logger = logging.getLogger(__name__)
SERPAPI_ENDPOINT = "https://serpapi.com/search.json"

# Standard schema used across all CF2 lead tools
CSV_FIELDS = [
    "name", "phone", "phone_formatted", "email", "website",
    "address", "location", "category", "source", "keyword",
    "rating", "review_count", "review_snippet",
    "destination_visited", "review_date", "hotel_reviewed",
    "intent_score", "quality_score", "segment", "last_verified",
]

def _load_api_key(credentials_file: str) -> str:
    """Load SerpAPI key from JSON credentials file (CF2 Standard Format)."""
    if not credentials_file: return ""
    p = Path(credentials_file).expanduser()
    if not p.exists(): return ""
    try:
        data = json.loads(p.read_text(encoding="utf-8"))
        key = str(data.get("api_key") or "").strip()
        if key: return key
        for k in data.get("keys", []):
            if k.get("status") == "active" and k.get("api_key"):
                return str(k["api_key"]).strip()
    return ""

```

```

except Exception:
    return ""

class LinkedInScraperInput(BaseModel):
    topic: str = Field(...)
    keywords: List[str] = Field(...)
    output_dir: str = Field(...)
    credentials_file: str = Field(default="")
    max_results_per_keyword: int = Field(default=20)
    skip_if_cached: bool = Field(default=True)

class LinkedInScraperTool(BaseTool):
    name: str = "leaddata_linkedin"
    description: str = "Scrape LinkedIn company directories for B2B leads."
    args_schema: Type[BaseModel] = LinkedInScraperInput

    def _run(self, topic: str, keywords: List[str], output_dir: str,
            credentials_file: str = "", max_results_per_keyword: int = 20,
            skip_if_cached: bool = True, **kwargs) -> str:

        out_dir = Path(output_dir) / "raw"
        out_dir.mkdir(parents=True, exist_ok=True)
        out_file = out_dir / "leads_raw.csv"
        api_key = _load_api_key(credentials_file)

        if not api_key:
            logger.error("❌ No SerpAPI key found for LinkedIn scrape.")
            return "❌ No SerpAPI key found."

        leads = []
        for kw in keywords[:5]: # Limit to prevent API exhaustion
            logger.info(f"🔍 Searching LinkedIn for: {kw[:60]}...")
            try:
                r = requests.get(SERPAPI_ENDPOINT, params={
                    "q": f'"{kw}" site:linkedin.com/company',
                    "api_key": api_key, "num": max_results_per_keyword, "hl":
"en"
                }, timeout=30)

                if r.status_code != 200:
                    logger.warning(f"⚠ HTTP {r.status_code} for {kw}")
                    continue

                results = r.json().get("organic_results", [])

                for res in results:
                    snippet = res.get("snippet", "")
                    link = res.get("link", "")
                    # Clean up the title (usually has " | LinkedIn" at the end)
                    title = res.get("title", "").replace(" | LinkedIn",
"").strip()

                    # Extract industry from snippet if possible
                    industry = "B2B Company"
                    industry_match = re.search(r'(\d+[,]? \d* \s* employees? | \bIT\b| \bFinancial\b| \bConstruction\b| \bSoftware\b)', snippet, re.IGNORECASE)
                    if industry_match: industry = industry_match.group(0)

                    leads.append({
                        "name": title,
                        "phone": "",
                        "phone_formatted": "",
                        "email": "",
                        "website": link,

```

```

        "address": "",
        "location": "",
        "category": industry,
        "source": "linkedin_company",
        "keyword": kw,
        "rating": 0,
        "review_count": 0,
        "review_snippet": snippet[:200],
        "destination_visited": "",
        "review_date": "",
        "hotel_reviewed": "",
        "intent_score": 20, # Low intent, high quality data
        "quality_score": "",
        "segment": "",
        "last_verified": time.strftime("%Y-%m-%d")
    })
    time.sleep(1.0)
except Exception as e:
    logger.error(f"❌ LinkedIn search error: {e}")

if not leads:
    logger.warning("⚠️ No LinkedIn companies found")
    return "⚠️ No LinkedIn companies found"

# Append to CSV
try:
    write_header = not out_file.exists() or out_file.stat().st_size == 0
    with open(out_file, 'a', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=CSV_FIELDS,
extrasaction='ignore')
        if write_header: w.writeheader()
        w.writerows(leads)
except Exception as e:
    logger.error(f"❌ Failed to write LinkedIn leads: {e}")
    return f"❌ Failed to write leads to CSV."

success_msg = f"✓ Added {len(leads)} LinkedIn companies"
logger.info(success_msg)
return success_msg

```

```

=====
=
"""
leaddata_news.py – Google News Trend Scraper (CF2 Tool)
Output: {output_dir}/raw/leads_raw.csv
"""
import logging
import csv
import time
import requests
from pathlib import Path
from typing import Type, List
from pydantic import BaseModel, Field
from crewai.tools import BaseTool

logger = logging.getLogger(__name__)
SERPAPI_ENDPOINT = "https://serpapi.com/search.json"
CSV_FIELDS = [
    "name", "phone", "phone_formatted", "email", "website",
    "address", "location", "category", "source", "keyword",
    "rating", "review_count", "review_snippet",
    "destination_visited", "review_date", "hotel_reviewed",
    "intent_score", "quality_score", "segment", "last_verified",
]

```

```

def _load_api_key(credentials_file: str) -> str:
    if not credentials_file: return ""
    p = Path(credentials_file).expanduser()
    if not p.exists(): return ""
    try:
        import json
        data = json.loads(p.read_text(encoding="utf-8"))
        key = str(data.get("api_key") or "").strip()
        if key: return key
        for k in data.get("keys", []):
            if k.get("status") == "active" and k.get("api_key"): return
    str(k["api_key"]).strip()
    return ""
    except Exception: return ""

class NewsTrendInput(BaseModel):
    topic: str = Field(...)
    keywords: List[str] = Field(...)
    output_dir: str = Field(...)
    credentials_file: str = Field(default="")
    max_results_per_keyword: int = Field(default=10)

class GoogleTrendsTool(BaseTool):
    name: str = "leaddata_trends"
    description: str = "Scrape Google News for trending articles related to the
topic."
    args_schema: Type[BaseModel] = NewsTrendInput

    def _run(self, topic: str, keywords: List[str], output_dir: str,
        credentials_file: str = "", max_results_per_keyword: int = 10,
        **kwargs) -> str:

        out_dir = Path(output_dir) / "raw"
        out_dir.mkdir(parents=True, exist_ok=True)
        out_file = out_dir / "leads_raw.csv"
        api_key = _load_api_key(credentials_file)

        if not api_key: return "✗ No SerpAPI key found."

        leads = []
        for kw in keywords[:3]:
            logger.info(f" Searching News for: {kw}")
            try:
                # Use tbm=nws for Google News
                r = requests.get(SERPAPI_ENDPOINT, params={
                    "q": kw, "engine": "google", "tbm": "nws",
                    "api_key": api_key, "num": max_results_per_keyword
                }, timeout=30)

                if r.status_code != 200: continue
                results = r.json().get("news_results", [])

                for res in results:
                    leads.append({
                        "name": res.get("source", "News Outlet"),
                        "phone": "", "phone_formatted": "", "email": "",
                        "website": res.get("link", ""),
                        "address": "", "location": "",
                        "category": "Trend/News", "source": "google_trends",
                        "keyword": kw, "rating": 0, "review_count": 0,
                        "review_snippet": res.get("title", ""),
                        "destination_visited": "", "review_date":
res.get("date", "")[:10],

```



```

        "hotel_reviewed": "", "intent_score": 10, # Very low
intent, high context
        "quality_score": "", "segment": "",
        "last_verified": time.strftime("%Y-%m-%d")
    })
    time.sleep(1.0)
except Exception as e:
    logger.error(f"News search error: {e}")

if not leads: return "⚠️ No news trends found"

write_header = not out_file.exists() or out_file.stat().st_size == 0
with open(out_file, 'a', newline='', encoding='utf-8') as f:
    w = csv.DictWriter(f, fieldnames=CSV_FIELDS, extrasaction='ignore')
    if write_header: w.writeheader()
    w.writerows(leads)

return f"✓ Added {len(leads)} trend/news signals"

```

```

=====
=
"""

```

leaddata_normalize.py – Normalize & Deduplicate Tool (Rule 16: single output)

Reads: {output_dir}/raw/leads_raw.csv

Writes: {output_dir}/normalized/leads_clean.csv

CRITICAL: Preserves intent_score and keyword columns for downstream scoring.

```

"""
import logging
from pathlib import Path
from typing import Type, List
import csv
import re
import hashlib
import unicodedata
from datetime import datetime, timezone
from pydantic import BaseModel, Field
from crewai.tools import BaseTool

logger = logging.getLogger(__name__)

# ⚠️ CRITICAL FIX: Added "keyword" and "intent_score" so they don't get deleted!
SCHEMA = [
    "name", "phone", "email", "website", "address",
    "location", "category", "source", "keyword", "intent_score", "last_verified"
]

def _norm_phone(p: str, country_default: str = "") -> str:
    if not p: return ""
    c = re.sub(r'^\d+', '', p.strip())
    if not c: return ""
    if c.startswith('+'): return c
    if country_default and country_default.startswith('+'): return
country_default + c
    return '+' + c

def _norm_url(u: str, force_https: bool = True) -> str:
    if not u: return ""
    u = u.strip()
    if u.startswith(('http://', 'https://')): return u
    return ('https://' if force_https else 'http://') + u

def _norm_text(t: str, strip_unicode: bool = True) -> str:

```

```

    if not t: return ""
    t = t.strip()
    if strip_unicode: t = unicodedata.normalize('NFD', t)
    return re.sub(r'\s+', ' ', t)

def _dedup_key(rec: dict, fields: List[str]) -> str:
    parts = []
    for f in fields:
        v = (rec.get(f, "") or "").lower().strip()
        if f == "phone": v = _norm_phone(v).lstrip('+')
        parts.append(v)
    s = '|'.join(parts)
    if not s.strip():
        return hashlib.md5((rec.get("name", "") or
    "").lower().encode()).hexdigest()[:16]
    return hashlib.md5(s.encode()).hexdigest()[:16]

class LeadDataNormalizeInput(BaseModel):
    output_dir: str = Field(...)
    deduplicate_on: List[str] = Field(default=["website"])
    phone_country_default: str = Field(default="")
    lowercase_email: bool = Field(default=True)
    force_https: bool = Field(default=True)
    strip_unicode: bool = Field(default=True)
    min_name_length: int = Field(default=2)

class LeadDataNormalizeTool(BaseTool):
    name: str = "leaddata_normalize"
    description: str = "Normalize and deduplicate leads. Output:
normalized/leads_clean.csv"
    args_schema: Type[BaseModel] = LeadDataNormalizeInput

    def _run(
        self,
        output_dir: str,
        deduplicate_on: List[str] = None,
        phone_country_default: str = "",
        lowercase_email: bool = True,
        force_https: bool = True,
        strip_unicode: bool = True,
        min_name_length: int = 2
    ) -> str:

        if deduplicate_on is None:
            deduplicate_on = ["website"]

        in_file = Path(output_dir) / "raw" / "leads_raw.csv"
        out_dir = Path(output_dir) / "normalized"
        out_dir.mkdir(parents=True, exist_ok=True)
        out_file = out_dir / "leads_clean.csv"

        if not in_file.exists():
            logger.error(f"Input missing: {in_file}")
            return f"❌ Input missing: {in_file}"

        with open(in_file, 'r', encoding='utf-8') as f:
            raw = list(csv.DictReader(f))

        ts = datetime.now(timezone.utc).isoformat()
        normalized = []

        for r in raw:
            name = _norm_text(r.get("name", "") or r.get("title", ""),
strip_unicode)

```

```

        if len(name) < min_name_length:
            continue

        email = (r.get("email", "") or "").strip()
        if lowercase_email:
            email = email.lower()

        # ⚠ CRITICAL FIX: Explicitly passing intent_score and keyword!
        normalized.append({
            "name": name,
            "phone": _norm_phone(r.get("phone", ""), phone_country_default),
            "email": email,
            "website": _norm_url(r.get("website", "") or r.get("link", "")),
force_https),
            "address": _norm_text(r.get("address", ""), strip_unicode),
            "location": _norm_text(r.get("location", ""), strip_unicode),
            "category": _norm_text(r.get("category", ""), strip_unicode),
            "source": r.get("source", "import"),
            "keyword": r.get("keyword", ""), # PRESERVED
            "intent_score": r.get("intent_score", 0), # PRESERVED
            "last_verified": ts,
        })

    # Deduplicate
    seen, unique = set(), []
    for rec in normalized:
        k = _dedup_key(rec, deduplicate_on)
        if k not in seen:
            seen.add(k)
            unique.append(rec)

    removed = len(normalized) - len(unique)

    try:
        with open(out_file, 'w', newline='', encoding='utf-8') as f:
            w = csv.DictWriter(f, fieldnames=SCHEMA, extrasaction='ignore')
            w.writeheader()
            w.writerows(unique)
    except Exception as e:
        logger.error(f"Failed to write normalized CSV: {e}")
        return f"❌ Failed to write output: {e}"

    logger.info(f"✓ Normalized {len(unique)} | Dedup removed {removed} → {out_file.name}")
    return f"✓ Normalized {len(unique)} | Dedup removed {removed} → {out_file.name}"

```

```

=====
=
"""

```

```

leaddata_reddit.py – Dynamic Reddit Intent Scraper (CF2 Tool)
Source: Any subreddits defined in config
Output: {output_dir}/raw/leads_raw.csv (appends to existing file)
Rule 16: Single output file per tool
Rule 32: Smart Skip mandatory
"""

```

```

import logging
import csv
import json
import time
import requests
from datetime import datetime
from pathlib import Path
from typing import Type, List, Set

```

```

from pydantic import BaseModel, Field
from crewai.tools import BaseTool

logger = logging.getLogger(__name__)

# Standard schema used across all CF2 lead tools
CSV_FIELDS = [
    "name", "phone", "phone_formatted", "email", "website",
    "address", "location", "category", "source", "keyword",
    "rating", "review_count", "review_snippet",
    "destination_visited", "review_date", "hotel_reviewed",
    "intent_score", "quality_score", "segment", "last_verified",
]

class RedditScrapeInput(BaseModel):
    topic: str = Field(...)
    keywords: List[str] = Field(...)
    output_dir: str = Field(...)
    credentials_file: str = Field(default="")
    subreddits: List[str] = Field(default=["smallbusiness", "Entrepreneur"])
    min_post_upvotes: int = Field(default=5) # Replaced karma check with post
score (faster)
    post_recency_days: int = Field(default=30)
    max_posts_per_sub: int = Field(default=50)
    skip_if_cached: bool = Field(default=True)

class RedditTravelScraperTool(BaseTool):
    name: str = "leaddata_reddit"
    description: str = "Scrape Reddit for high-intent discussions on ANY topic."
    args_schema: Type[BaseModel] = RedditScrapeInput

    def _run(
        self,
        topic: str,
        keywords: List[str],
        output_dir: str,
        credentials_file: str = "",
        subreddits: List[str] = None,
        min_post_upvotes: int = 5,
        post_recency_days: int = 30,
        max_posts_per_sub: int = 50,
        skip_if_cached: bool = True,
        **kwargs
    ) -> str:

        if not subreddits:
            subreddits = ["smallbusiness"]

        out_dir = Path(output_dir) / "raw"
        out_dir.mkdir(parents=True, exist_ok=True)
        out_file = out_dir / "leads_raw.csv"

        # — Rule 32: Smart Skip —————
        cache_key = f"reddit_{','.join(subreddits)}_{post_recency_days}d"
        cache_file = out_dir / f".cache_{cache_key}.json"

        if skip_if_cached and cache_file.exists():
            try:
                cache = json.loads(cache_file.read_text())
                if (datetime.now() -
datetime.fromisoformat(cache["timestamp"])).days < 1:
                    logger.info(f"🔴 Skipped Reddit scrape (cached):

```

```

{len(cache['leads'])} leads"
        return f"🔴 Skipped Reddit scrape (cached):
{len(cache['leads'])} leads"
    except Exception:
        pass

leads = []
headers = {"User-Agent": "CF2-LeadData-System/1.0"}
search_query = " OR ".join(f'"{kw}"' for kw in keywords)

for sub in subreddits:
    logger.info(f"🔍 Scraping r/{sub} for: {search_query[:80]}...")

    try:
        r = requests.get(
            f"https://www.reddit.com/r/{sub}/search.json",
            params={
                "q": search_query,
                "sort": "new",
                "limit": max_posts_per_sub,
                "restrict_sr": "true",
                "type": "link"
            },
            headers=headers,
            timeout=30
        )

        # Handle Reddit API rate limits
        if r.status_code == 429:
            logger.warning(f"⚠️ Reddit rate limit hit on r/{sub}.
Sleeping 10s...")
            time.sleep(10)
            continue
        if r.status_code != 200:
            logger.warning(f"⚠️ Reddit API error: HTTP {r.status_code}")
            continue

        posts = r.json().get("data", {}).get("children", [])

        for post in posts:
            data = post.get("data", {})

            # Filter by recency
            created_utc = datetime.fromtimestamp(data.get("created_utc",
0))
            if (datetime.now() - created_utc).days > post_recency_days:
                continue

            # Filter by post quality (upvotes instead of user karma to
avoid API calls)
            if data.get("score", 0) < min_post_upvotes:
                continue

            author = data.get("author", "")
            if not author or author == "[deleted]":
                continue

            post_url = f"https://reddit.com{data.get('permalink', '')}"
            title = data.get("title", "")
            selftext = data.get("selftext", "")

            leads.append({
                "name": author,
                "phone": "",

```

```

        "phone_formatted": "",
        "email": "",
        "website": post_url,
        "address": "",
        "location": "",
        "category": topic,
        "source": "reddit_planner",
        "keyword": sub,
        "rating": data.get("score", 0), # Using score as rating
        "review_count": data.get("num_comments", 0),
        "review_snippet": f"{title} {selftext}"[:200],
        "destination_visited": "",
        "review_date": created_utc.isoformat(),
        "hotel_reviewed": "",
        "intent_score": 85, # Active posting = high intent
        "quality_score": "",
        "segment": "",
        "last_verified": datetime.now().strftime("%Y-%m-%d"),
    })

except requests.exceptions.Timeout:
    logger.error(f"⌚ Timeout scraping r/{sub}")
except Exception as e:
    logger.error(f"⚠️ Error scraping r/{sub}: {e}")
    continue

if not leads:
    logger.warning("⚠️ No Reddit leads found matching criteria")
    return "⚠️ No Reddit leads found matching criteria"

# — Deduplicate by Post URL (not username) —————
existing_urls = set()
if out_file.exists():
    try:
        with open(out_file, 'r', encoding='utf-8') as f:
            for row in csv.DictReader(f):
                if row.get("source") == "reddit_planner" and
row.get("website"):
                    existing_urls.add(row["website"])
    except Exception:
        pass

new_leads = [l for l in leads if l["website"] not in existing_urls]

# — Write to CSV —————
try:
    write_header = not out_file.exists() or out_file.stat().st_size == 0
    with open(out_file, 'a', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=CSV_FIELDS,
extrasaction='ignore')
        if write_header:
            writer.writeheader()
        writer.writerows(new_leads)
except Exception as e:
    logger.error(f"❌ Failed to write Reddit leads: {e}")
    return f"❌ Failed to write leads to CSV."

# — Save Cache —————
try:
    cache_file.write_text(json.dumps({
        "timestamp": datetime.now().isoformat(),
        "leads": [l["website"] for l in new_leads]
    }))
except Exception:

```

pass

```
success_msg = f"✓ Added {len(new_leads)} Reddit leads to leads_raw.csv"
logger.info(success_msg)
return success_msg
```

```
=====
=
"""
```

leaddata_score.py – Dynamic Score & Segment Tool (Rule 16: single output)

```
Reads: {output_dir}/normalized/leads_clean.csv
Writes: {output_dir}/scored/leads_scored.csv
```

Enhanced: Automatically normalizes ANY scoring rubric passed from YAML.
(e.g., if YAML says "intent_score": 40, it scales the 0-100 intent score to a 0-40 weight).

```
"""
import logging
from pathlib import Path
from typing import Type, Dict, Any
import csv
from pydantic import BaseModel, Field
from crewai.tools import BaseTool

# Initialize logger
logger = logging.getLogger(__name__)

# Fallback rubric if YAML doesn't provide one
DEFAULT_RUBRIC = {
    "has_phone": 20, "has_email": 20, "has_website": 20,
    "has_address": 20, "active_business": 20,
}

# Output schema ensures intent_score and keyword are preserved for downstream
tools
SCHEMA_OUT = [
    "name", "phone", "email", "website", "address",
    "location", "category", "source", "keyword", "intent_score",
    "quality_score", "segment", "last_verified"
]

def _score(rec: dict, rubric: dict) -> int:
    total_score = 0
    is_intent_lead = rec.get("source", "") in ["intent_osint", "reddit_planner"]

    for key, weight in rubric.items():
        val = rec.get(key)

        # Handle numeric columns (like intent_score: 85)
        if isinstance(val, (int, float)):
            max_baseline = 100 if "score" in key.lower() else 10
            normalized_val = min(val / max_baseline, 1.0)
            total_score += int(normalized_val * weight)

        # Handle text/boolean columns (like phone, email)
        elif isinstance(val, str):
            if val.strip():
                total_score += weight

    elif bool(val):
        total_score += weight
```

```

# ⚠ CRITICAL FIX: If this is an intent lead but has 0 intent score,
# it means it's garbage (e.g., "What is glamping?"). Kill the score.
if is_intent_lead:
    try:
        intent_val = int(rec.get("intent_score", 0))
        if intent_val == 0:
            return 0 # Force to Cold
    except (ValueError, TypeError):
        pass

return min(total_score, 100)

def _segment(score: int, t: dict) -> str:
    if score >= t.get("hot", 70): return "hot"
    if score >= t.get("warm", 40): return "warm"
    return "cold"

class LeadDataScoreInput(BaseModel):
    output_dir: str = Field(...)
    score_enabled: bool = Field(default=True)
    scoring_rubric: Dict[str, int] = Field(default_factory=dict)
    thresholds: Dict[str, int] = Field(default_factory=lambda: {
        "hot": 70, "warm": 40, "cold": 0
    })
    sort_by_score_desc: bool = Field(default=True)

class LeadDataScoreTool(BaseTool):
    name: str = "leaddata_score"
    description: str = "Score and segment leads dynamically based on YAML
    rubric. Output: scored/leads_scored.csv"
    args_schema: Type[BaseModel] = LeadDataScoreInput

    def _run(
        self,
        output_dir: str,
        score_enabled: bool = True,
        scoring_rubric: Dict[str, int] = None,
        thresholds: Dict[str, int] = None,
        sort_by_score_desc: bool = True
    ) -> str:

        # Use YAML rubric if provided, else fallback to default
        rubric = scoring_rubric if scoring_rubric else DEFAULT_RUBRIC
        if thresholds is None:
            thresholds = {"hot": 70, "warm": 40, "cold": 0}

        in_file = Path(output_dir) / "normalized" / "leads_clean.csv"
        out_dir = Path(output_dir) / "scored"
        out_dir.mkdir(parents=True, exist_ok=True)
        out_file = out_dir / "leads_scored.csv"

        if not in_file.exists():
            logger.error(f"Input missing: {in_file}")
            return f"❌ Input missing: {in_file}"

        # Read input data
        try:
            with open(in_file, 'r', encoding='utf-8') as f:
                records = list(csv.DictReader(f))
        except Exception as e:
            logger.error(f"Failed to read {in_file}: {e}")

```



```

        return f"❌ Failed to read CSV: {e}"

    if not records:
        logger.warning("No records found to score.")
        return "⚠️ No records found to score."

    logger.info(f"Scoring {len(records)} records using rubric:
{list(rubric.keys())}")

    counts = {"hot": 0, "warm": 0, "cold": 0, "skipped": 0}

    for r in records:
        if not score_enabled:
            score = 0
        else:
            score = _score(r, rubric)

        r["quality_score"] = score
        r["segment"] = _segment(score, thresholds)
        counts[r["segment"]] += 1

    # Ensure intent_score is preserved (default to 0 if missing from
normalize step)
    if "intent_score" not in r or not r["intent_score"]:
        r["intent_score"] = 0

    # Sort by quality
    if sort_by_score_desc:
        records.sort(key=lambda x: int(x.get("quality_score", 0)),
reverse=True)

    # Write output
    try:
        with open(out_file, 'w', newline='', encoding='utf-8') as f:
            # Use extrasaction='ignore' so any extra columns from intent/raw
don't crash the writer
            w = csv.DictWriter(f, fieldnames=SCHEMA_OUT,
extrasaction='ignore')
            w.writeheader()
            w.writerows(records)
    except Exception as e:
        logger.error(f"Failed to write {out_file}: {e}")
        return f"❌ Failed to write output: {e}"

    logger.info(f"Scored {len(records)} | 🔥{counts['hot']}
◆{counts['warm']} ❄️{counts['cold']}")
    return (
        f"✓ Scored {len(records)} | "
        f"🔥{counts['hot']} ◆{counts['warm']} ❄️{counts['cold']} →
{out_file.name}"
    )

```

```

=====
=

```

```

{
  "topic": "need equipment financing Canada, looking for equipment loan Ontario,
how to finance a dump truck Ontario, best equipment loan for construction
Canada, heavy machinery financing options Alberta",
  "_version": "1.0.0",
  "_profile": "leads",
  "_comment": "CF2 – Targeting BORROWERS (construction/trucking), not LENDERS.",
  "Unit-LeadData": true,

```

```

"Unit-Scout":      false,
"Unit-Data":      false,
"Unit-Debate":    false,
"Unit-Prodcast":  false,
"Unit-Classroom": false,
"Unit-Definition": false,
"Unit-Animation": false,
"Unit-Comparison": false,
"Unit-Packaging": false,
"Unit-Publisher": false,
"Unit-Advertise": false,

"leaddata_config": {
  "enabled": true,
  "enrich_enabled": false,

  "location_sources": [],
  "directory_sources": ["linkedin_company"],
  "community_sources": ["reddit_business", "quora"],
  "trend_sources": ["google_trends"],

  "credentials_file": "serpapi_credentials.json",
  "skip_if_cached": false,

  "tool_overrides": {
    "reddit_business": {
      "force_reddit": true,
      "max_results_per_keyword": 20
    },
    "quora": {
      "force_quora": true,
      "max_results_per_keyword": 20
    }
  },

  "normalize_config": {
    "deduplicate_on": ["website"],
    "lowercase_email": true,
    "min_name_length": 2
  },

  "score_config": {
    "score_enabled": true,
    "scoring_rubric": {
      "source": 40,
      "intent_score": 40,
      "review_count": 15,
      "review_date": 5
    },
    "segment_thresholds": { "hot": 60, "warm": 35, "cold": 0 },
    "sort_by_score_desc": true
  },

  "export_config": {
    "formats": ["csv", "json"],
    "generate_stats": true,
    "stats_file": "lead_stats.json",
    "include_segments_breakdown": true
  }
}
}

```

```

=====
=

```

input/profile/leadint.json

```
{
  "topic": "Plan tour WestJet vacation,Air Transat vacation",
  "_comment": "CF2 – Config-driven AI automation. Topic-focused pipeline
execution.",
  "_profile": "leadint",
  "_version": "1.0.0",

  "Unit-LeadData": true,
  "Unit-Scout": false,
  "Unit-Data": false,
  "Unit-Debate": false,
  "Unit-Prodcast": false,
  "Unit-Classroom": false,
  "Unit-Definition": false,
  "Unit-Animation": false,
  "Unit-Comparison": false,
  "Unit-Packaging": false,
  "Unit-Publisher": false,
  "Unit-Advertise": false,

  "leaddata_config": {
    "enabled": true,
    "enrich_enabled": true,
    "enrich_config": {
      "max_enrich_rows": 50,
      "allow_guessing": true,
      "min_confidence": 0.0,
      "skip_if_cached": true,
      "max_osint_queries": 60,
      "query_delay_seconds": 1
    },
    "location_sources": ["maps_reviewers"],
    "directory_sources": [],
    "community_sources": [],
    "trend_sources": [],

    "credentials_file": "serpapi_credentials.json",
    "skip_if_cached": true,

    "tool_overrides": {
      "maps_reviewers": {
        "review_recency_days": 7,
        "min_reviewer_activity": 1
      }
    },
    "normalize_config": {
      "deduplicate_on": ["name"],
      "phone_country_default": "US",
      "lowercase_email": true,
      "force_https": true,
      "strip_unicode": true,
      "min_name_length": 2
    },
    "score_config": {
      "score_enabled": true,
      "scoring_rubric": {
        "source": 40,
        "intent_score": 40,
```

```
        "review_count": 15,  
        "review_date": 5  
    },  
    "segment_thresholds": {  
        "hot": 60,  
        "warm": 35,  
        "cold": 0  
    },  
    "sort_by_score_desc": true  
},  
  
"export_config": {  
    "formats": ["csv", "json"],  
    "generate_stats": true,  
    "stats_file": "lead_stats.json",  
    "include_segments_breakdown": true  
}  
}  
}
```

=====

=